

1 Introdução

[TODO]

1.1 Objetivos

[TODO]

1.1.1 Objetivo Geral

[TODO]

1.1.2 Objetivos Específicos

[TODO]

1.2 Organização da monografia

[TODO]

Este trabalho é composto por XX capítulos. Neste Capítulo 1, proporcionamos uma introdução, justificativa e delineamos os objetivos...

2 Fundamentação Teórica

Este capítulo apresenta os conceitos necessários para a compreensão do trabalho. Inicialmente, são introduzidos fundamentos de aprendizado profundo e redes neurais artificiais, seguidos pelo processo de treinamento dessas redes. Em seguida, discute-se o problema do consumo energético em modelos de *Deep Learning* e, por fim, são apresentadas as técnicas de compressão de modelos, com destaque para a poda neural (*pruning*), que é o foco deste trabalho.

2.1 Aprendizado Profundo e Redes Neurais

2.1.1 Aprendizado Profundo (Deep Learning)

O Aprendizado Profundo é um campo especializado de aprendizado de máquina (GOODFELLOW; BENGIO; COURVILLE, 2016), caracterizado pelo uso de redes neurais artificiais com múltiplas camadas, o que permite aprender e extrair informações complexas a partir de grandes volumes de dados.

2.1.2 Redes Neurais Artificiais

A rede neural artificial é um modelo computacional inspirado na estrutura do cérebro humano, que busca imitar a forma como neurônios se conectam e processam sinais para aprender a partir de dados, identificando padrões e tomando decisões. Um exemplo clássico é o perceptron multicamadas (MLP), um modelo composto por funções matemáticas que mapeiam valores de entrada em valores de saída (GOODFELLOW; BENGIO; COURVILLE, 2016). Em tarefas de visão computacional, por exemplo, milhões de imagens de um objeto podem ser processadas para que a rede aprenda suas características visuais e seja capaz de reconhecê-lo posteriormente.

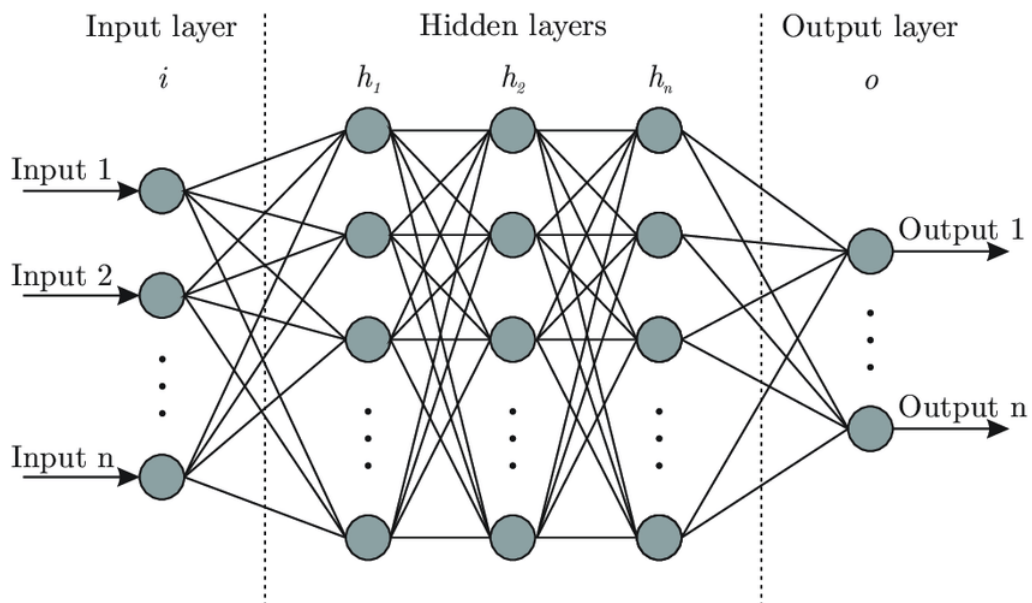
Uma rede neural profunda é composta por unidades (nós) interconectadas, organizadas em camadas.

- **Nó (neurônio):** É uma unidade computacional que calcula uma soma ponderada das entradas mais um *bias* e passa o resultado através de uma função de ativação, produzindo uma saída.
- **Camada de Entrada (Input Layer):** É onde o processo é iniciado e os dados são introduzidos na rede. O número de nós nessa camada corresponde ao número de características dos dados de entrada.

- **Camadas Ocultas (Hidden Layers):** São camadas intermediárias entre a entrada e a saída. Cada nó de uma camada oculta está conectado a todos os nós das camadas anterior e seguinte. É nelas que ocorre a maior parte do processamento e do aprendizado; a saída de uma camada oculta é utilizada como entrada da próxima, e, a cada camada, a rede passa a detectar características mais complexas e abstratas. Redes com múltiplas camadas ocultas são as chamadas redes de *Deep Learning*.
- **Camada de Saída (Output Layer):** Camada final da rede, onde o resultado é produzido. A estrutura da camada de saída depende da tarefa: em regressão, há um único nó com função de ativação linear; em classificação binária, um único nó expressa resultados do tipo sim/não; em classificação multiclasse, há um nó para cada classe possível, e o de maior ativação corresponde à predição final.

Cada conexão entre nós possui um peso associado, que determina a sua importância relativa. Pesos altos amplificam o sinal de entrada, enquanto pesos próximos de zero tornam a conexão pouco influente. Os pesos são inicializados com valores aleatórios e ajustados durante o treinamento.

Figura 1 – Arquitetura típica de uma rede neural artificial com uma camada de entrada, duas camadas ocultas e uma camada de saída.



2.1.3 Bias em Redes Neurais

O *bias* é um parâmetro adicional que ajusta a saída de um neurônio. Trata-se de um valor numérico associado a cada neurônio, não ligado a nenhuma entrada específica, atuando como um coeficiente linear. Antes da aplicação da função de ativação, o *bias* é somado ao resultado da soma ponderada das entradas.

A principal função do *bias* é deslocar a função de ativação, permitindo que o modelo se ajuste melhor aos dados. Sem ele, a soma ponderada precisaria ser alta para que o neurônio fosse ativado; com o *bias*, um neurônio pode ser ativado mesmo com uma soma ponderada baixa, caso seu *bias* seja suficientemente alto. Inversamente, um *bias* muito negativo dificulta a ativação do neurônio. O *bias* é um parâmetro aprendível: inicialmente é inicializado com zero ou valores próximos de zero e, durante o treinamento, é atualizado pelo mesmo processo de retropropagação e descida de gradiente utilizado para os pesos.

2.1.4 Função de Ativação

A função de ativação é uma função matemática aplicada ao resultado da soma ponderada das entradas de um neurônio (já acrescido do *bias*), determinando a saída do neurônio. Sua principal função é introduzir não linearidade no modelo: sem ela, a composição de diversas camadas resultaria em uma transformação puramente linear, equivalente a uma rede com uma única camada, o que limitaria drasticamente a capacidade expressiva do modelo (GOODFELLOW; BENGIO; COURVILLE, 2016).

A ativação de um neurônio ocorre em três etapas:

- Soma ponderada: o neurônio multiplica o valor de ativação de cada nó da camada anterior pelo peso da respectiva conexão e soma todos os resultados.
- Adição do *bias*: ao resultado da soma ponderada é somado o valor do *bias*, que auxilia o neurônio a ativar-se mais ou menos facilmente.
- Função de ativação: o resultado final (soma ponderada acrescida do *bias*) é passado pela função de ativação, cuja saída é então transmitida aos nós da próxima camada.

Figura 2 – Representação matricial da propagação de um neurônio, da soma ponderada à função de ativação (Sigmoide).

02_capitulo/imageMatrizes_sigmoid.png

Há diversas funções de ativação, cada uma com suas próprias características e aplicações.

- **ReLU (Rectified Linear Unit):** Função de ativação mais utilizada em redes neurais modernas, proposta para redes profundas por Nair e Hinton (2010) e popularizada por Krizhevsky, Sutskever e Hinton (2012). É uma função linear por partes, com inclinação 1 para valores positivos e 0 para valores negativos. É computacionalmente muito eficiente e não sofre do problema do desaparecimento do gradiente

para entradas positivas. Em contrapartida, pode sofrer do problema conhecido como *Dying ReLU*, no qual neurônios cujas entradas se tornam persistentemente negativas deixam de ser ativados e param de aprender.

- **Sigmoide:** Função que mapeia qualquer valor real para o intervalo $(0, 1)$, sendo útil em classificação binária, já que a saída pode ser interpretada como uma probabilidade. Caiu em desuso em camadas ocultas porque, para entradas com magnitude muito alta ou muito baixa, sua derivada se aproxima de zero, fenômeno conhecido como desaparecimento do gradiente.
- **Tanh (Tangente Hiperbólica):** Semelhante à Sigmoide, porém mapeando os valores para o intervalo $(-1, 1)$, o que a torna centrada em zero e costuma favorecer a convergência. Também sofre do problema do desaparecimento do gradiente.
- **Softmax:** Diferentemente das demais, não é aplicada a neurônios individuais de camadas ocultas, mas sim ao conjunto dos neurônios da camada de saída em tarefas de classificação multiclasse. Transforma as saídas brutas da última camada em uma distribuição de probabilidades, com valores no intervalo $(0, 1)$ que somam 1.

Em cada camada, as ativações produzidas determinam as ativações da camada seguinte. De camada em camada, esses padrões tornam-se progressivamente mais complexos e abstratos, até que a camada de saída produz o resultado final. Assim, camadas iniciais aprendem características simples, enquanto camadas mais profundas combinam essas características para representar conceitos de alto nível.

2.2 Treinamento de Redes Neurais

2.2.1 Gradiente

No contexto de redes neurais, o gradiente corresponde ao vetor de derivadas parciais da função de perda em relação aos pesos da rede. Ele é o elemento central que permite a minimização da função de perda por meio do algoritmo da descida do gradiente, um método de primeira ordem que utiliza as informações do gradiente para atualizar os parâmetros da rede (GOODFELLOW; BENGIO; COURVILLE, 2016).

Cada componente do vetor de gradiente é uma derivada parcial, calculada assumindo que as demais variáveis são mantidas constantes. A magnitude da derivada indica o quanto a função está variando; o seu sinal, a direção dessa variação: positivo, a função está aumentando; negativo, diminuindo.

Uma analogia comum é a de um caminhante que desce uma encosta em meio à neblina, enxergando apenas o solo ao seu redor: a cada passo, ele sente para qual lado o terreno mais desce (o oposto do gradiente) e avança nessa direção. Na descida do

gradiente, a “encosta” é a função de perda e os “passos” são os ajustes nos pesos, repetidos até alcançar um ponto de mínimo.

2.2.2 Backpropagation

A retropropagação (*backpropagation*) é o algoritmo que permite o aprendizado das redes neurais, por possibilitar o cálculo da contribuição de cada parâmetro (peso ou *bias*) para o erro da predição (RUMELHART; HINTON; WILLIAMS, 1986). Uma vez que o erro é calculado na camada de saída, o algoritmo executa um fluxo invertido: da saída em direção à entrada, propagando o erro para as camadas anteriores.

Para saber como cada peso individual se correlaciona com o erro, o algoritmo calcula a derivada parcial da função de perda em relação a esse peso, utilizando a regra da cadeia multivariada para decompor o cálculo em derivadas intermediárias. Com todas as derivadas obtidas, um algoritmo de otimização atualiza cada peso na direção oposta ao gradiente:

$$w_{novo} = w_{antigo} - \eta \cdot \frac{\partial L}{\partial w}$$

em que η é a taxa de aprendizado, hiperparâmetro que controla o tamanho do ajuste aplicado a cada iteração.

2.2.3 Hiperparâmetros

Hiperparâmetros são variáveis externas que definem tanto o algoritmo de aprendizado quanto a arquitetura da rede neural. Diferentemente dos parâmetros do modelo (pesos e *biases*), que são ajustados automaticamente pelo *backpropagation*, os hiperparâmetros são definidos antes do início do treinamento e controlam como a rede aprende a partir dos dados. Existem técnicas automatizadas para sua busca, como *grid search* e otimização bayesiana, mas o ajuste manual baseado em experiência ainda é amplamente empregado.

Em geral, os hiperparâmetros podem ser agrupados em duas categorias:

- **Hiperparâmetros de arquitetura:** definem a estrutura da rede, como o número de camadas ocultas, o número de neurônios em cada camada e o tipo de função de ativação utilizada.
- **Hiperparâmetros de treinamento:** controlam como o modelo se ajusta aos dados, como a taxa de aprendizado, o número de épocas, o tamanho do lote (*batch size*) e a escolha do algoritmo de otimização.

2.2.4 Passo a passo do treinamento

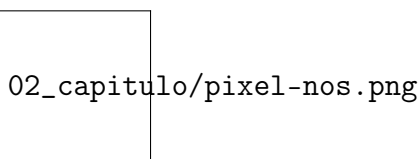
O treinamento de uma rede neural supervisionada requer um grande conjunto de dados rotulados, isto é, cada exemplo de entrada deve estar associado a uma saída correta. Esse conjunto é tradicionalmente dividido em três subconjuntos (GOODFELLOW; BENGIO; COURVILLE, 2016):

- **Conjunto de treinamento**, usado para ajustar os pesos e *biases*;
- **Conjunto de validação**, usado para monitorar o desempenho durante o treinamento e ajustar hiperparâmetros;
- **Conjunto de teste**, usado para avaliar o desempenho final do modelo de forma imparcial.

Além disso, é necessário definir a arquitetura da rede (camadas, neurônios, funções de ativação) e os hiperparâmetros de treinamento. O treinamento ocorre ao longo de diversas iterações, nas quais cada passagem completa pelo conjunto de treinamento é chamada de época (*epoch*); em cada época, os dados são processados em pequenos lotes (*batches*).

Como exemplo, considere uma rede neural que classifica imagens de 28×28 pixels. A camada de entrada terá 784 nós, cada um recebendo o valor de intensidade de um pixel — ou seja, uma característica do dado de entrada.

Figura 3 – Mapeamento entre pixels de uma imagem 28×28 e os nós da camada de entrada de uma rede neural.



O ciclo de treinamento pode ser resumido nos seguintes passos:

- **Passo 1 – Inicialização dos parâmetros:** pesos são inicializados com valores aleatórios baixos e *biases* com zero. Essa quebra de simetria permite que diferentes neurônios aprendam características distintas.
- **Passo 2 – *Forward pass*:** um lote de dados é inserido na camada de entrada e propagado através das camadas ocultas até a camada de saída, produzindo uma predição.
- **Passo 3 – Cálculo da perda:** a predição é comparada com o rótulo verdadeiro por meio de uma função de perda, que quantifica numericamente o quão distante a predição está do valor correto.

- **Passo 4 – *Backpropagation*:** o erro é propagado de volta pela rede, calculando-se o gradiente da função de perda em relação a cada peso e *bias*.
- **Passo 5 – Atualização dos parâmetros:** um otimizador atualiza pesos e *biases* na direção oposta ao gradiente, reduzindo a função de perda.
- **Passo 6 – Repetição:** os passos 2 a 5 são repetidos para cada lote e ao longo de um número definido de épocas.

O treinamento é interrompido quando o modelo atinge uma precisão satisfatória ou quando a função de perda deixa de diminuir significativamente.

2.3 Arquitetura Transformer

A arquitetura *Transformer*, proposta por Vaswani et al. (2017) no trabalho *Attention is All You Need*, marcou uma mudança profunda na forma como modelos de aprendizado profundo processam sequências de texto. Antes dessa proposta, o campo era dominado por arquiteturas que processavam a sequência elemento por elemento, palavra por palavra. O *Transformer* rompeu com essa abordagem ao introduzir um mecanismo capaz de considerar toda a sequência de uma só vez. Esta seção apresenta, de forma progressiva, os conceitos necessários para compreender a arquitetura utilizada neste trabalho: as limitações das abordagens anteriores, o mecanismo de atenção, a operação de autoatenção e sua versão multi-cabeça, a estrutura de cada bloco da rede e, finalmente, o modelo GPT-2, que é o objeto central dos experimentos de poda.

2.3.1 Limitações das Arquiteturas Recorrentes

Até a chegada do *Transformer*, o processamento de texto era dominado pelas redes neurais recorrentes (*Recurrent Neural Networks*, RNNs), popularizadas ao longo da década de 1990. Diferentemente de arquiteturas puramente diretas, as RNNs processam a sequência palavra por palavra e reaproveitam a saída de cada passo como entrada do passo seguinte, criando uma forma de “memória”: a cada nova palavra, um estado oculto (*hidden state*) é atualizado com base na palavra atual e no estado anterior, de modo que, ao final, ele deve resumir as informações relevantes de todo o texto.

Essa memória, porém, falha em sequências longas. Considere o trecho “A Inglaterra é minha terra natal. Passei toda a minha vida lá. Mudei-me para a Espanha há dois dias. Eu falo apenas uma língua, que é o ____”. Para completar a lacuna corretamente com “inglês”, o modelo precisa recuperar a palavra “Inglaterra”, que aparece no início do trecho. O obstáculo é o chamado gradiente desaparecente (*vanishing gradient*): durante o treinamento via *backpropagation*, os gradientes são multiplicados repetidamente

pelas matrizes de peso da rede e, quando esses valores são pequenos, encolhem até se aproximarem de zero (BENGIO; SIMARD; FRASCONI, 1994). Com isso, a influência de “Inglaterra” torna-se insignificante ao chegar à lacuna, e o modelo perde o contexto necessário — limitação que se manifesta como incapacidade de lidar com **dependências de longo alcance**.

Há ainda uma segunda limitação, de ordem computacional: como o estado oculto no instante t só pode ser calculado após o do instante $t - 1$, o processamento é estritamente sequencial e mal aproveita o paralelismo das unidades de processamento gráfico (GPUs) modernas. Essas duas dificuldades impulsionaram uma evolução cronológica de arquiteturas. As redes LSTM (*Long Short-Term Memory*), propostas em 1997, atacaram o gradiente desaparecente por meio de portas (*gates*) que regulam o que deve ser mantido ou esquecido, permitindo reter por mais tempo informações como “Inglaterra” (HOCHREITER; SCHMIDHUBER, 1997). Ainda assim, as LSTMs continuam processando a frase de forma sequencial, o que as mantém lentas e difíceis de paralelizar em grandes volumes de dados — gargalo que só seria superado pelo mecanismo de atenção.

2.3.2 Mecanismo de Atenção

O passo seguinte dessa evolução foi o mecanismo de atenção (*attention mechanism*), introduzido por Bahdanau, Cho e Bengio (2015) no contexto da tradução automática. Em vez de comprimir toda a frase em um único estado oculto, a atenção permite que o modelo “olhe” de volta e foque seletivamente em partes específicas da entrada. No exemplo anterior, ela pode consultar diretamente o termo “Inglaterra”, independentemente de quão distante ele esteja da lacuna, eliminando o risco de o gradiente desaparecer pelo caminho.

Para operacionalizar essa ideia, o mecanismo utiliza três conceitos: a consulta (*query*), a chave (*key*) e o valor (*value*). Uma analogia útil é a de uma biblioteca: cada livro tem uma etiqueta na lombada (a *chave*) que descreve seu conteúdo; quando um leitor chega com uma pergunta (a *consulta*), ela é comparada com cada etiqueta para decidir quais livros são mais relevantes; o conteúdo desses livros (os *valores*) é então combinado, com os mais relevantes contribuindo mais para a resposta. No *Transformer*, esse processo ocorre com números: a relevância entre uma consulta e cada chave é calculada matematicamente, e os valores são somados com pesos proporcionais a essa relevância.

A inovação central de Vaswani et al. (2017), em 2017, foi construir uma arquitetura inteira baseada exclusivamente nesse mecanismo, dispensando por completo a recorrência. Com isso, todas as posições de uma sequência se comunicam diretamente em uma única operação, sem precisar “passar” por palavras intermediárias. Isso resolve as duas limitações das RNNs de uma só vez: as dependências de longo alcance tornam-se tão fáceis de capturar quanto as de curto alcance, e o cálculo passa a ser paralelizável, tornando as

arquiteturas baseadas em RNN e LSTM obsoletas para muitas tarefas modernas.

2.3.3 Autoatenção e Atenção Multi-Cabeça

A variante do mecanismo de atenção utilizada no *Transformer* é chamada de autoatenção (*self-attention*). O prefixo “auto” indica que a consulta, a chave e o valor são todos derivados da mesma sequência de entrada. Ou seja, ao processar uma frase, cada palavra pergunta a si mesma: “quais outras palavras desta frase são relevantes para minha interpretação?”. Exemplo: “O gato que estava no telhado dormiu”. Ao processar a palavra “dormiu”, o mecanismo de autoatenção calcula um peso para cada palavra da frase. A palavra “gato” recebe peso alto, pois é o sujeito do verbo; palavras como “telhado” ou “no” recebem pesos baixos, pois são menos relevantes para determinar quem dormiu.

Na prática, esses pesos são calculados da seguinte forma, a partir de cada representação de entrada, três matrizes de pesos aprendíveis durante o treinamento transformam cada representação em três vetores: o vetor de consulta Q (*query*), o vetor de chave K (*key*) e o vetor de valor V (*value*). O grau de compatibilidade entre uma consulta e cada chave é calculado pelo produto escalar entre eles. Esses valores brutos de compatibilidade passam pela função *softmax*, que os converte em uma distribuição de probabilidades, isto é, um conjunto de números entre 0 e 1 que somam exatamente 1. Esses pesos normalizados determinam com que intensidade cada valor V contribui para o resultado final, que é obtido pela soma ponderada dos valores.

Essa operação completa é denominada *Scaled Dot-Product Attention* (atenção por produto escalar escalado) e é definida formalmente por (VASWANI et al., 2017):

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V$$

Em termos intuitivos, o produto $QK^\top$ mede a compatibilidade entre todas as consultas e todas as chaves de uma só vez; a divisão por $\sqrt{d_k}$ (a dimensão das chaves) é um fator de escala que estabiliza o treinamento; a *softmax* converte essas pontuações em pesos que somam 1; e a multiplicação por V produz, para cada posição, uma média ponderada dos valores, isto é, sua representação já contextualizada pelo restante da sequência.

Uma única operação de atenção, porém, captura apenas um tipo de relação entre as palavras de cada vez. Para superar essa limitação, Vaswani et al. (2017) propõem a atenção multi-cabeça (*multi-head attention*). A analogia é a seguinte: se a atenção simples é como um único leitor relendo a frase em busca de relações gramaticais, a atenção multi-cabeça é como h leitores independentes, cada um procurando um tipo diferente de relação. Um leitor pode focar em concordância verbal (“gato”/“dormiu”), outro em relações de posse (“telhado do vizinho”), outro em referências pronominais, e assim por diante. O GPT-2 *small*, por exemplo, utiliza 12 cabeças de atenção por bloco.

Na prática, cada uma das h cabeças executa a operação *Scaled Dot-Product Attention* de forma independente, com suas próprias matrizes de projeção aprendíveis para Q , K e V . As saídas das h cabeças são depois concatenadas e projetadas por uma matriz linear aprendível, produzindo a representação final de cada posição. Desse modo, o modelo pode simultaneamente capturar relações em diferentes subespaços semânticos e sintáticos, sem custo adicional expressivo em relação à atenção simples.

Sob a perspectiva da compressão de modelos, as cabeças de atenção constituem unidades podáveis naturais para a poda estruturada: estudos da literatura demonstram que nem todas as cabeças são igualmente úteis, e que algumas podem ser removidas inteiramente sem degradação significativa da qualidade das previsões (MICHEL; LEVY; NEUBIG, 2019).

2.3.4 Estrutura do Bloco Transformer

Para entender como o *Transformer* processa informação em profundidade, é útil pensar em cada bloco como uma “estação de trabalho” em uma linha de montagem. Na primeira bancada, as palavras trocam informações entre si por meio da atenção multi-cabeça: cada palavra atualiza sua representação consultando todas as demais. Na segunda bancada, cada palavra refina individualmente sua própria representação, de forma independente das outras, por meio de uma pequena rede totalmente conectada. Esse par de operações se repete em cada bloco, e os blocos são empilhados, fazendo com que as representações das palavras se tornem progressivamente mais ricas e abstratas a cada andar.

O primeiro bloco é a camada de atenção multi-cabeça, já apresentada na seção anterior, responsável por integrar informação contextual de toda a sequência. O segundo é uma rede *feedforward* totalmente conectada — um MLP (*multilayer perceptron*) aplicado de forma idêntica e independente a cada posição. Esse MLP é composto por duas transformações lineares com uma função de ativação não linear entre elas: a função GELU (*Gaussian Error Linear Unit*) (HENDRYCKS; GIMPEL, 2016) nos modelos GPT, ou ReLU em outras variantes. Diferentemente da ReLU (NAIR; HINTON, 2010), que simplesmente zera valores negativos, a GELU faz uma transição suave em torno da origem. Na prática, essa suavidade evita o problema dos “neurônios mortos” (*dying ReLU*), em que unidades deixam de aprender por receberem gradiente nulo, e por isso Radford et al. (2019) a adotaram como ativação padrão na família GPT. A dimensão intermediária desse MLP é tipicamente quatro vezes maior do que a dimensão do modelo — no GPT-2 *small*, a dimensão do modelo é 768 e a dimensão intermediária é 3.072 — o que confere ao bloco grande capacidade de representação antes de comprimir novamente a informação.

2.3.5 Modelos de Linguagem de Grande Escala e o GPT-2

Modelos de linguagem de grande escala (*Large Language Models*, LLMs) são redes neurais profundas, tipicamente baseadas na arquitetura *Transformer*, treinadas em grandes volumes de texto com o objetivo de aprender a prever palavras. A tarefa de pré-treinamento é simples de enunciar: dado um trecho de texto, o modelo deve prever qual token vem a seguir. Um *token* é a unidade mínima de texto que o modelo processa — pode ser uma palavra, parte de uma palavra, um sinal de pontuação ou um espaço. Dada a sequência “O aluno abriu o”, um LLM bem treinado atribuiria alta probabilidade a tokens como “livro”, “caderno” ou “computador”, e baixa probabilidade a tokens como “foguetes” ou “plutônio”. Repetida bilhões de vezes sobre textos variados, essa tarefa aparentemente simples força o modelo a aprender gramática, fatos sobre o mundo, raciocínio básico e muito mais (RADFORD et al., 2019).

Para que o texto possa ser processado numericamente, ele precisa primeiro ser convertido em tokens e depois em vetores numéricos. Essa conversão ocorre em duas etapas. A primeira é a tokenização, realizada por algoritmos como o BPE (*Byte Pair Encoding*), que divide o texto em unidades de comprimento variável de forma a equilibrar cobertura e eficiência. Por exemplo, a palavra “inconstitucional” pode ser dividida em tokens como “in”, “constitu” e “cional”. O GPT-2 possui um vocabulário de aproximadamente 50.257 tokens. A segunda etapa converte cada token em um vetor numérico denso por meio de uma camada de *embedding*: o modelo aprende, durante o treinamento, uma representação vetorial para cada token do vocabulário, de modo que tokens semanticamente próximos tendam a ter representações próximas no espaço vetorial.

O GPT-2 (*Generative Pre-trained Transformer 2*), proposto por Radford et al. (2019) pela OpenAI em 2019, é um modelo autorregressivo baseado exclusivamente no componente decodificador do *Transformer* original. Ele foi pré-treinado em um corpus de aproximadamente 40 GB de texto coletado da internet, abrangendo uma grande diversidade de domínios e estilos. A variante GPT-2 *small*, utilizada neste trabalho, possui 12 blocos *Transformer* empilhados, 12 cabeças de atenção por bloco, dimensão de modelo igual a 768 e aproximadamente 124 milhões de parâmetros treináveis. Apesar de ser a menor variante da família GPT-2, esse número de parâmetros é cerca de 70 vezes maior do que o de uma ResNet-50 (25 milhões de parâmetros), o que ilustra a escala característica dos modelos baseados em *Transformer*.

O crescimento em escala dos LLMs ao longo dos anos seguintes foi vertiginoso: o GPT-3, publicado em 2020, possui 175 bilhões de parâmetros; o PaLM, do Google, chega a 540 bilhões. Mesmo modelos de médio porte como o GPT-2 impõem custos energéticos e computacionais consideráveis em ambientes de produção, onde a inferência é executada milhares ou milhões de vezes por dia. A poda estruturada surge, nesse contexto, como uma estratégia concreta para reduzir esses custos: a remoção de cabeças de atenção

redundantes e de neurônios MLP de baixa importância pode diminuir substancialmente a contagem de operações por inferência, com impacto controlado sobre a qualidade das predições (PATTERSON et al., 2021).

2.4 Consumo Energético em Redes Neurais

O avanço de técnicas e *hardware* voltados ao treinamento de redes neurais, motivado pela busca por maior precisão, trouxe consigo um consumo de energia considerável e, conseqüentemente, uma significativa emissão de dióxido de carbono. Strubell, Ganesh e McCallum (2019) relatam, por exemplo, que o treinamento de um modelo *transformer* com busca de arquitetura neural pode emitir cerca de 626.155 libras de CO_2 , enquanto a emissão média de um automóvel ao longo de toda a sua vida útil é de aproximadamente 126.000 libras — cerca de cinco vezes menos.

Esse custo energético concentra-se em duas etapas do ciclo de vida de um modelo: o treinamento e a inferência.

No **treinamento**, o modelo é alimentado com grandes volumes de dados e ajustado iterativamente pelo *backpropagation*. Esse processo é computacionalmente intenso e geralmente requer *hardware* especializado (GPUs ou TPUs), muitas vezes organizado em *clusters* com centenas ou milhares de aceleradores.

Na **inferência**, o modelo já treinado é utilizado para gerar predições. O custo de uma única predição é relativamente baixo; contudo, em contextos de larga escala — como os modelos de IA generativa servindo milhões de requisições diárias — o custo energético agregado é expressivo (LUCCIONI; VIGUIER; LIGOZAT, 2022).

2.5 Compressão de Modelos

Com o crescimento do *Deep Learning*, os modelos de IA vêm dobrando em tamanho a cada poucos anos (SEVILLA et al., 2022). Modelos de alta performance costumam ser otimizados exclusivamente para acurácia, resultando em redes com grande quantidade de parâmetros, que ocupam muito espaço de armazenamento e demandam um elevado número de operações em ponto flutuante (FLOPs) para serem executadas. Isso é particularmente problemático em dispositivos com capacidade limitada, como *smartphones* e sistemas embarcados, e também em contextos de uso em larga escala, nos quais o custo energético se torna significativo.

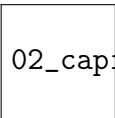
A compressão de modelos é um campo que reúne técnicas voltadas a atenuar esses problemas, reduzindo o tamanho dos modelos e/ou a quantidade de operações necessárias para sua execução, preservando, tanto quanto possível, a acurácia. As redes neurais

modernas possuem alta redundância de parâmetros (DENIL et al., 2013), o que torna viável remover ou simplificar aqueles com baixa contribuição ao resultado final.

As quatro principais famílias de técnicas de compressão são:

- **Quantização (Quantization):** reduz o número de *bits* utilizados para representar cada peso do modelo. Em vez de pontos flutuantes de 32 bits, os parâmetros passam a ser representados com precisão reduzida (16 ou 8 bits, por exemplo), o que diminui o tamanho do modelo e pode acelerar operações em *hardware* que suporta aritmética de menor precisão (JACOB et al., 2018).
- **Destilação de Conhecimento (Knowledge Distillation):** proposta por Hinton, Vinyals e Dean (2015), é uma metodologia de treinamento em que um modelo menor (*aluno*) aprende a partir de um modelo maior e mais preciso (*professor*). O aluno é treinado não apenas para acertar os rótulos verdadeiros, mas também para imitar a distribuição de saída do professor, resultando em um modelo compacto que, frequentemente, alcança acurácia superior à que teria se fosse treinado do zero.
- **Fatoração de Matrizes (Matrix Factorization):** matrizes de pesos de grandes dimensões são decompostas em produto de matrizes menores cuja multiplicação aproxima a matriz original, reduzindo o custo computacional.
- **Poda Neural (Neural Pruning):** processo de remoção de pesos — ou de estruturas maiores, como filtros — que contribuem pouco para a saída do modelo. É a técnica central deste trabalho e será detalhada na Seção 2.6.

Figura 4 – Ilustração do processo de poda: a rede original (densa) tem conexões e/ou neurônios removidos, tornando-se esparsa.



02_capitulo/imagepoda.png

2.6 Poda Neural (Pruning)

A poda neural é uma técnica de compressão que visa reduzir o tamanho do modelo e o custo computacional da inferência por meio da remoção de parâmetros que pouco contribuem para o resultado final. O processo mais comum parte de um modelo previamente treinado, cujo foco inicial foi exclusivamente a acurácia, e aplica a poda como uma etapa posterior.

2.6.1 Tipos de Poda

Na literatura, duas categorias principais de poda são consideradas (BLALOCK et al., 2020):

- **Poda não estruturada:** remoção de pesos individuais, independentemente da sua posição na matriz de pesos. Esta abordagem oferece maior flexibilidade e, geralmente, taxas de compressão elevadas com perda reduzida de acurácia. Entretanto, do ponto de vista do custo de inferência, a poda não estruturada apresenta uma limitação importante: processadores e GPUs convencionais não são eficientes em pular multiplicações por zero espalhadas de forma aleatória na matriz de pesos. Assim, embora o modelo tenha muitos zeros, as operações ainda são executadas, e o ganho prático de velocidade e energia tende a ser marginal, a menos que se utilize *hardware* especializado ou formatos de armazenamento esparsos.
- **Poda estruturada:** remoção de estruturas inteiras, como neurônios, filtros ou canais convolucionais, o que implica a remoção completa de detectores de características. Como o resultado é uma matriz menor e ainda densa, há um ganho direto de velocidade e eficiência em *hardware* convencional. O custo dessa abordagem é uma potencial perda maior de acurácia para uma mesma taxa de compressão.

Cabe mencionar também a chamada **esparsidade estruturada** (por exemplo, padrões 2:4), em que o algoritmo é forçado a zerar, a cada bloco de quatro pesos, exatamente dois. Como o *hardware* sabe previamente quantos e em que formato os zeros aparecem, a matriz pode ser comprimida de maneira determinística, permitindo acelerações reais em GPUs modernas que suportam esse padrão (NVIDIA Corporation, 2020).

Assim, a poda estruturada é a que apresenta benefícios mais imediatos em termos de FLOPs e consumo de energia em *hardware* convencional, razão pela qual ela recebe destaque neste trabalho.

2.6.2 Poda por Magnitude

A técnica mais simples e amplamente estudada é a poda por magnitude. Nela, considera-se que pesos cujos valores absolutos são próximos de zero contribuem pouco para a saída do modelo e, portanto, podem ser removidos. Define-se um limiar (ou uma taxa de esparsidade desejada) e todos os pesos abaixo desse limiar são zerados, criando redes esparsas (HAN et al., 2015).

2.6.3 Pipeline Geral da Poda

De forma geral, a poda pode ser dividida nos seguintes passos:

Figura 5 – Pipeline clássico de poda por magnitude: treinamento do modelo denso, remoção dos pesos de menor magnitude e retreinamento (*fine-tuning*) dos pesos remanescentes.

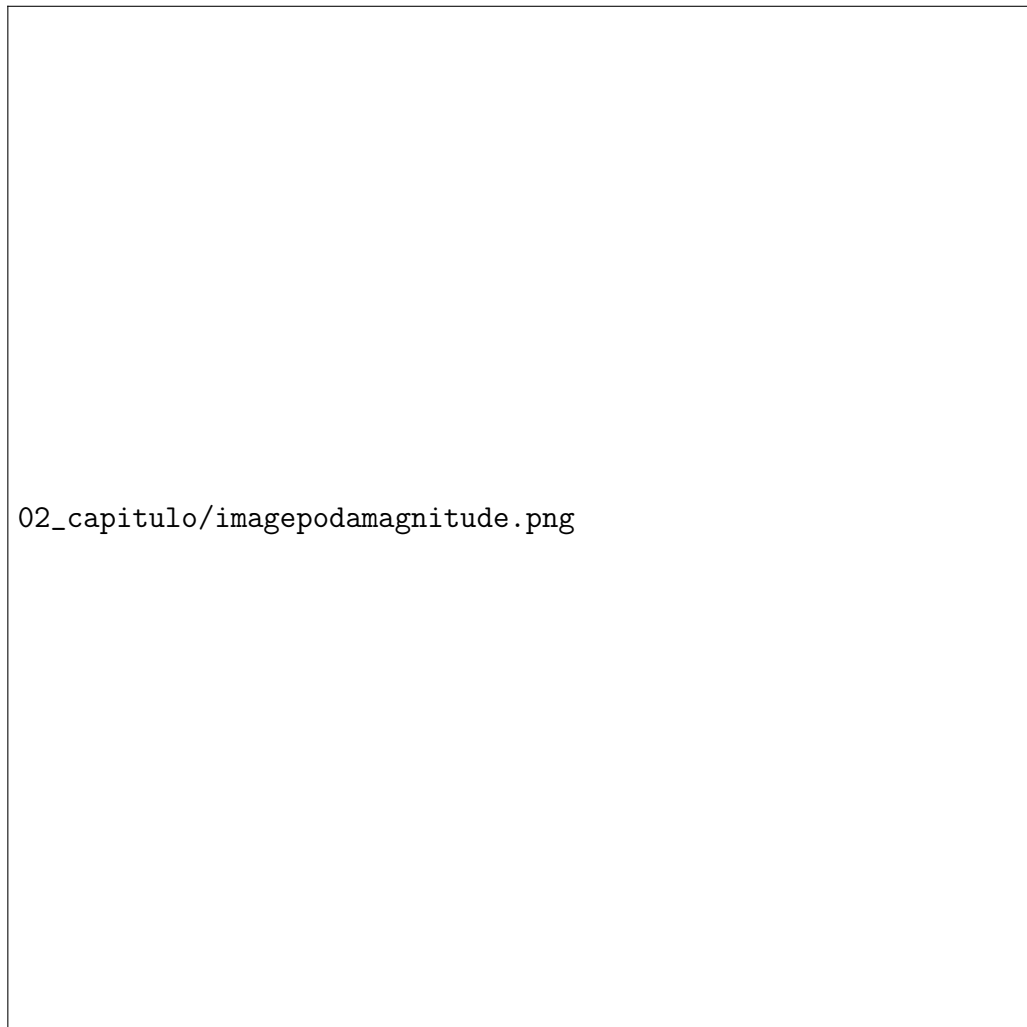


Figura 6 – Comparação esquemática entre uma rede densa e uma rede podada, em que conexões de menor importância são removidas.

02_capitulo/imagepruning.png

- **Passo 1 – Definição de critério e taxa de poda:** partindo de um modelo já treinado, define-se o critério de importância dos parâmetros (magnitude absoluta, norma L_1 de filtros, sensibilidade, entre outros) e a taxa de esparsidade desejada.
- **Passo 2 – Identificação dos parâmetros a podar:** com base no critério escolhido, o algoritmo classifica os parâmetros por importância e seleciona os menos relevantes.
- **Passo 3 – Execução da poda:** os parâmetros selecionados são zerados (no caso da poda não estruturada) ou fisicamente removidos (no caso da poda estruturada),

gerando um modelo esparso ou uma nova arquitetura reduzida.

- **Passo 4 – Ajuste fino (*fine-tuning*):** como a remoção tende a deteriorar a acurácia, o modelo podado é retreinado com uma taxa de aprendizado reduzida, para que os parâmetros remanescentes compensem a ausência dos que foram removidos.
- **Passo 5 – Iteração:** a poda pode ser aplicada de forma iterativa, em pequenas etapas intercaladas por *fine-tuning*, o que costuma preservar melhor a acurácia do que uma poda agressiva em um único passo.

2.6.4 Quando a Poda é Aplicada

Tradicionalmente, a poda é aplicada após o treinamento inicial do modelo, seguida de um ajuste fino. Métodos mais recentes aplicam a poda de forma gradual durante o próprio treinamento, intercalando passos de treino e poda, o que permite à rede adaptar-se progressivamente à nova estrutura.

Uma linha de pesquisa mais recente investiga a possibilidade de podar a rede antes do treinamento, identificando sub-redes inerentemente promissoras dentro de uma rede densa inicializada aleatoriamente. Essa abordagem é impulsionada pela chamada *Lottery Ticket Hypothesis* (FRANKLE; CARBIN, 2019), segundo a qual, dentro de uma rede grande inicializada aleatoriamente, existe uma sub-rede pequena (o “bilhete premiado”) que, quando treinada isoladamente com a mesma inicialização, atinge acurácia comparável à da rede completa.

3 Metodologia

Este capítulo apresenta o percurso metodológico adotado no trabalho, organizado em cinco etapas. As decisões concretas decorrentes desses critérios — incluindo o modelo selecionado, os *datasets* empregados e as configurações de poda — são descritas no Capítulo 4.

3.1 Seleção e Implementação do Modelo de Referência

O modelo de referência (*baseline*) corresponde à versão original do modelo de linguagem de grande escala (*Large Language Model* — LLM) antes de qualquer processo de poda. Ele serve como padrão para mensurar a perda de desempenho introduzida pela compressão: todas as métricas dos modelos podados são comparadas às métricas desse modelo original.

Para a seleção do modelo de referência, foram adotados os seguintes critérios:

- ser um LLM representativo, amplamente utilizado na literatura de compressão de modelos;
- dispor de pesos pré-treinados publicamente disponíveis;
- apresentar tamanho compatível com as restrições de *hardware* disponível;
- possuir ampla documentação em trabalhos de poda neural;
- contar com implementação acessível em *frameworks* consolidados, como PyTorch.

A implementação do modelo de referência utiliza a biblioteca PyTorch (PASZKE et al., 2019). Como repositório de modelos pré-treinados, é empregada a plataforma Hugging Face, um ambiente colaborativo que disponibiliza ampla variedade de modelos de linguagem de grande escala publicamente acessíveis — como, por exemplo, variantes do GPT-2. O modelo selecionado a partir dos critérios estabelecidos — incluindo a versão específica obtida via Hugging Face — é apresentado e detalhado no Capítulo 4.

3.2 Implementação das Estratégias de Poda sem Otimização

As estratégias de poda sem otimização têm por objetivo reduzir o número de parâmetros de um modelo pré-treinado, eliminando pesos ou estruturas avaliadas como menos relevantes para o desempenho preditivo. Serão avaliadas duas estratégias de poda

clássicas, escolhidas por representarem os dois extremos do espectro de granularidade da poda neural:

- poda baseada em magnitude dos pesos (HAN et al., 2015), que remove parâmetros individuais cujos valores absolutos são inferiores a um limiar; representa a granularidade mais fina possível e gera modelos esparsos sem alteração da estrutura física do modelo;
- poda estruturada (LI et al., 2017), que elimina unidades arquiteturais inteiras — como filtros convolucionais ou cabeças de atenção — reduzindo fisicamente as dimensões dos tensores e gerando modelos computacionalmente mais enxutos.

Ambas as estratégias são amplamente consolidadas na literatura de compressão de modelos neurais, com mais de uma década de aplicação e extensa validação experimental. Sua inclusão neste trabalho não constitui contribuição metodológica original; pelo contrário, sua aplicação fornece uma base de comparação estabelecida, sobre a qual serão avaliadas as estratégias otimizadas propostas na Seção 3.3.

Os critérios que orientaram a escolha das estratégias a implementar foram:

- representatividade na literatura de compressão de modelos;
- complementaridade entre granularidades distintas de remoção;
- viabilidade de implementação com as ferramentas disponíveis.

Em ambas as estratégias, são aplicadas funções de avaliação de importância de parâmetros, que orientam o processo de remoção: a poda por magnitude utiliza o valor absoluto dos pesos como critério, enquanto a poda estruturada utiliza a norma L_1 dos filtros (ou unidades arquiteturais equivalentes).

As estratégias selecionadas com base nesses critérios, bem como os detalhes de sua implementação, são apresentados no Capítulo 4.

3.3 Implementação das Estratégias de Poda com Otimização

Esta é a etapa central do trabalho. O objetivo é investigar se a combinação das estratégias de poda com técnicas de otimização resulta em modelos mais eficientes do que a poda aplicada de forma direta, sem qualquer refinamento adicional.

As estratégias de otimização a investigar incluem:

- Comparação entre taxas de poda fixas e dinâmicas;

- Variação do momento de aplicação da poda — antes do treinamento, durante ou após o *fine-tuning*;
- Análise de sensibilidade por camada, para alocação não uniforme do orçamento de poda;
- *Knowledge Distillation*, na qual um modelo menor (*student*) é treinado para reproduzir o comportamento de um modelo maior (*teacher*).

A comparação entre os modelos obtidos com e sem otimização visa identificar combinações que melhorem o balanço entre desempenho preditivo e eficiência computacional em relação às estratégias aplicadas sem otimização.

3.4 Métricas de Avaliação

As métricas de avaliação são organizadas em três dimensões: desempenho preditivo, custo computacional e impacto ambiental. Para cada métrica, descreve-se o que ela representa e como é obtida operacionalmente.

Desempenho preditivo

A métrica de desempenho preditivo adotada depende da natureza da tarefa do modelo selecionado. Em tarefas de classificação, a métrica usual é a acurácia — definida como a proporção de predições corretas sobre o total de amostras avaliadas — ou medidas derivadas, como precisão, revocação e F1 (GOODFELLOW; BENGIO; COURVILLE, 2016). Em modelos de linguagem voltados à geração de texto, a métrica padrão é a *perplexity*, que quantifica o grau de “surpresa” do modelo diante de cada *token* do conjunto de validação, constituindo uma medida da qualidade da modelagem probabilística da linguagem. Valores mais baixos de *perplexity* indicam melhor capacidade preditiva.

A métrica específica adotada neste trabalho será apresentada no Capítulo 4, em coerência com a tarefa do modelo selecionado.

Custo computacional

O custo computacional é avaliado por meio de quatro indicadores complementares:

- **Número de parâmetros remanescentes:** contagem direta dos tensores de pesos não-zerados (no caso de poda não estruturada) ou existentes (no caso de poda estruturada) no modelo após a aplicação da técnica;

- **Operações de ponto flutuante (FLOPs):** contagem das operações matemáticas necessárias para uma inferência completa, obtida por análise estática do grafo computacional do modelo. Bibliotecas específicas — como `thop`, em ambiente PyTorch — automatizam essa contagem percorrendo cada camada do modelo e acumulando as operações correspondentes;
- **Tempo de inferência:** medido empiricamente, registrando o tempo decorrido durante o processamento de uma entrada representativa em condições controladas de *hardware*. Múltiplas medições são agregadas por meio de média e desvio-padrão para mitigar a variância natural de execução;
- **Tamanho do modelo:** avaliado em duas dimensões — espaço ocupado em disco após serialização do estado do modelo e consumo de memória durante a inferência, este último registrado por ferramentas de *profiling* do PyTorch.

Impacto ambiental

O impacto ambiental é mensurado por meio de dois indicadores:

- **Consumo energético:** estimado por meio de ferramentas de monitoramento do uso de *hardware* (CPU e GPU) durante a execução dos experimentos. A biblioteca CodeCarbon (COURTY et al., 2023) integra leituras periódicas de consumo elétrico dos componentes, registrando a energia total consumida (em kWh) durante o intervalo de inferência;
- **Emissões equivalentes de CO₂:** calculadas a partir do consumo energético registrado, multiplicado por um fator de intensidade carbônica da matriz energética da região onde os experimentos são executados. A biblioteca CodeCarbon mantém base atualizada desses fatores e realiza essa conversão automaticamente.

4 Desenvolvimento

Este capítulo descreve, a metodologia adotada para o desenvolvimento experimental deste trabalho. São apresentados o *dataset* e o modelo selecionados, o protocolo de treinamento do *baseline*, o *framework* de implementação, os dois métodos de poda da literatura que serão avaliados, a estratégia otimizada proposta, o *pipeline* experimental, as métricas utilizadas, os critérios de comparação e as principais limitações metodológicas reconhecidas.

4.1 Visão Geral

Este capítulo descreve o percurso metodológico adotado para investigar em que medida diferentes abordagens de poda neural reduzem o consumo energético de redes convolucionais profundas e qual o *trade-off* correspondente em acurácia. A pergunta de pesquisa que orienta o trabalho é, portanto, de natureza empírica e comparativa: não se busca demonstrar que a poda é eficiente em termos absolutos — resultado já estabelecido pela literatura (HAN et al., 2015; LI et al., 2017; BLALOCK et al., 2020) — mas sim quantificar, sob condições controladas e reproduzíveis, como diferentes granularidades de remoção (não estruturada *versus* estruturada) afetam métricas de eficiência energética real, e se uma estratégia de alocação adaptativa de taxas por camada consegue melhorar esse balanço.

A abordagem adotada é estritamente empírica. Partindo de um modelo de referência obtido por *transfer learning* — uma ResNet-50 pré-treinada em ImageNet-1K e ajustada (*fine-tuned*) no CIFAR-100 — são aplicados dois métodos de poda consolidados na literatura, avaliados sob um conjunto comum de métricas que inclui acurácia, complexidade computacional (FLOPs), tamanho do modelo, latência de inferência e consumo energético empírico medido com a ferramenta **CodeCarbon** (COURTY et al., 2023). Em seguida, é desenvolvida uma estratégia otimizada, baseada em análise de sensibilidade por camada, cujos resultados são comparados sistematicamente com os dos métodos anteriores. Toda a implementação utiliza o *framework* PyTorch (PASZKE et al., 2019), e cada configuração experimental é repetida com três sementes aleatórias distintas para garantir a estabilidade das estimativas reportadas.

A escolha por um protocolo comparativo — em vez de otimizar apenas um único método — reflete a compreensão de que o impacto da poda sobre o consumo energético não é homogêneo entre paradigmas. A poda não estruturada, ao anular pesos individualmente sem alterar a estrutura do modelo, gera esparsidade lógica que, em *hardware* convencional, não se traduz necessariamente em redução do tempo de inferência ou do consumo de

energia. A poda estruturada, ao remover filtros inteiros e reduzir as dimensões dos tensores intermediários, produz modelos fisicamente menores e operacionalmente mais eficientes. Colocar ambos os paradigmas sob avaliação comum é essencial para responder à pergunta de pesquisa com evidências concretas.

4.1.1 As Quatro Etapas Centrais

O experimento é organizado em quatro etapas sequenciais, cada uma com papel distinto no desenho do trabalho.

A primeira etapa consiste no treinamento do modelo de referência (*baseline*): uma ResNet-50 inicializada com pesos pré-treinados em ImageNet-1K é submetida a *fine-tuning* no CIFAR-100, com todos os seus parâmetros ajustados à nova tarefa de classificação em 100 classes. Esse modelo denso é avaliado no conjunto de teste e tem todas as suas métricas registradas — acurácia, tamanho, FLOPs, latência e consumo energético. Ele serve como ponto de referência absoluto contra o qual todos os modelos podados serão comparados; sem um *baseline* sólido e rastreável, qualquer afirmação sobre ganhos de eficiência perde o referencial.

A segunda etapa aplica dois métodos de poda da literatura, em sequência independente a partir do mesmo *baseline*: a poda não estruturada por magnitude (HAN et al., 2015) e a poda estruturada por norma L_1 de filtros (LI et al., 2017). Para cada método, são testadas quatro taxas de poda (20%, 40%, 60% e 80%), e cada modelo resultante passa por *fine-tuning* com um protocolo de recuperação de acurácia antes de ser avaliado. A seleção desses dois métodos é deliberada: eles representam os dois extremos do espectro de granularidade de poda e têm comportamentos opostos em termos de impacto real sobre o *hardware*. A comparação direta entre eles é, por si só, uma das contribuições empíricas do trabalho.

A terceira etapa introduz a estratégia otimizada proposta neste trabalho: a análise de sensibilidade por camada. Em vez de aplicar uma taxa global uniforme, cada camada convolucional da ResNet-50 é podada individualmente em diferentes taxas, e a queda de acurácia resultante é registrada sem *fine-tuning*. Esse perfil de sensibilidade guia a alocação de taxas por camada, preservando camadas críticas e podando de forma mais agressiva aquelas com maior redundância. A expectativa é que essa alocação adaptativa permita atingir níveis de compressão comparáveis aos dos métodos uniformes com menor sacrifício de acurácia — hipótese que será testada experimentalmente e discutida no Capítulo 5.

A quarta etapa é a comparação sistemática. Todos os modelos produzidos — *baseline*, modelos podados pelos dois métodos da literatura em quatro taxas, e modelos da estratégia otimizada — são avaliados pelo mesmo conjunto de métricas, no mesmo

ambiente de *hardware* e com o mesmo conjunto de teste. Essa simetria de avaliação é condição necessária para que as comparações sejam válidas e interpretáveis. Os resultados são reportados como média e desvio-padrão sobre as três repetições com sementes distintas.

4.1.2 Discussão das Decisões Metodológicas

Todo desenho experimental implica escolhas que condicionam o escopo e a generalidade dos resultados. A Tabela 1 sintetiza as principais decisões metodológicas adotadas neste trabalho, as alternativas consideradas e a justificativa de cada escolha. As subseções a seguir desenvolvem em maior profundidade as decisões mais críticas para a interpretação dos resultados.

Tabela 1 – Síntese das principais decisões metodológicas do trabalho.

Decisão	Alternativas consideradas	Escolha	Justificativa
<i>Dataset</i>	ImageNet-1K, CIFAR-10, CIFAR-100	CIFAR-100	Equilíbrio entre complexidade da tarefa e custo computacional.
Modelo	ResNet-50, VGG-16, MobileNet	ResNet-50	Padrão da literatura de poda; arquitetura representativa.
Pré-treinamento	Treinar do zero; <i>transfer learning</i>	<i>Transfer learning</i> (ImageNet)	Custo computacional viável; consistente com a prática real.
Critério de poda	Magnitude, ativação, gradiente, segunda ordem	Norma L_1 (peso e filtro)	Simplicidade e ampla cobertura na literatura.
Granularidade	Não estruturada; estruturada	Ambas (Métodos 1 e 2)	A comparação entre granularidades é a contribuição empírica do trabalho.
Taxas testadas	Lineares, logarítmicas, adaptativas	20%, 40%, 60%, 80%	Coerente com trabalhos de referência (HAN et al., 2015; LI et al., 2017).
<i>Fine-tuning</i> pós-poda	Sem retreino; com retreino	Sim (10 épocas)	Padrão estabelecido na literatura para recuperar acurácia.
Estratégia otimizada	Mesma taxa por camada; taxas adaptativas	Taxas por camada (sensibilidade)	Permite alocar a poda onde há mais redundância.
Repetições	1, 3, 5, 10 <i>seeds</i>	3 <i>seeds</i>	Mínimo viável para média e desvio-padrão.
Conjunto de avaliação	Validação; teste	Teste	Avaliação final, sem viés de ajuste de hiperparâmetros.

Fonte: Elaboração própria.

A escolha do CIFAR-100 como *dataset* principal merece justificativa detalhada, pois é uma das decisões que mais condiciona o escopo e a generalidade dos resultados. O ImageNet-1K, embora seja o padrão-ouro em benchmarks de classificação visual e a fonte dos pesos pré-treinados utilizados neste trabalho, excede os objetivos deste estudo: o foco aqui não é reproduzir um benchmark de larga escala, mas sim analisar comparativamente o efeito de diferentes métodos de poda sobre acurácia, consumo energético e eficiência computacional — análise que não exige a escala do ImageNet-1K para ser conclusiva. O CIFAR-10, no extremo oposto, com apenas dez classes, oferece complexidade insuficiente para que as diferenças de acurácia entre modelos podados e não podados sejam estatisticamente expressivas. O CIFAR-100 ocupa um ponto de equilíbrio técnico: 100 classes mutuamente exclusivas e imagens naturais de baixa resolução tornam a tarefa suficientemente desafiadora para que os efeitos da poda se manifestem de forma mensurável, ao mesmo tempo que o tamanho do conjunto (60.000 imagens) permite a realização de múltiplas repetições experimentais, necessárias para a obtenção de estimativas estáveis das métricas de energia. A ampla adoção do CIFAR-100 em trabalhos de poda (LI et al., 2017; FRANKLE; CARBIN, 2019; BLALOCK et al., 2020) tem a vantagem adicional de facilitar comparações diretas com a literatura.

A ResNet-50 foi selecionada por reunir três propriedades relevantes para os objetivos do trabalho. Em primeiro lugar, trata-se de uma arquitetura representativa e madura, cujos comportamentos sob poda são amplamente documentados (LI et al., 2017; HE et al., 2016), o que permite contextualizar os resultados obtidos. Em segundo lugar, a presença de *skip connections* nos blocos residuais introduz restrições de dimensionalidade não triviais durante a poda estruturada, tornando esse modelo um banco de testes realista para avaliar a capacidade das ferramentas de implementação — em particular, a biblioteca *Torch-Pruning* (FANG et al., 2023), que gerencia automaticamente a propagação dos cortes pelo grafo de dependências. Em terceiro lugar, com aproximadamente 25,6 milhões de parâmetros e uma profundidade de 50 camadas, a ResNet-50 é suficientemente grande para que as diferenças de consumo energético entre configurações de poda sejam mensuráveis, sem ser tão grande a ponto de tornar os experimentos computacionalmente proibitivos. Arquiteturas projetadas explicitamente para eficiência não foram consideradas como modelo principal porque já incorporam, por projeto, mecanismos implícitos de compressão (convoluções separáveis, gargalos invertidos, escalonamento composto), o que confundiria a análise do efeito isolado da poda sobre um modelo denso convencional. Modelos baseados em atenção (*Vision Transformers*), por sua vez, foram descartados por uma razão distinta: os métodos de poda estruturada para *transformers* são ainda recentes e não consolidaram um repertório comparável ao disponível para CNNs clássicas, o que prejudicaria a comparação direta com os trabalhos de referência adotados (HAN et al., 2015; LI et al., 2017).

A comparação entre os dois métodos de poda — não estruturado e estruturado —

é necessária para responder à pergunta de pesquisa de forma completa. Um trabalho que avaliasse apenas a poda estruturada poderia concluir que a poda reduz o consumo energético, mas não saberia se essa redução se deve à granularidade da remoção ou simplesmente ao fato de o modelo ter menos parâmetros. A inclusão da poda não estruturada como controle permite isolar esse efeito: como ambos os métodos removem a mesma proporção de parâmetros (em taxa percentual equivalente), mas apenas o método estruturado reduz efetivamente os FLOPs e as operações executadas em *hardware* convencional, a diferença observada nas métricas de energia e latência pode ser atribuída à granularidade da remoção, e não à redução da magnitude do modelo em si. Essa distinção é central para a interpretação dos resultados e para as recomendações práticas que emergem do trabalho.

A estratégia otimizada proposta — poda estruturada com taxas por camada determinadas por análise de sensibilidade — parte da hipótese de que diferentes camadas de uma mesma rede apresentam sensibilidades distintas à remoção de filtros (HAN et al., 2015; YANG; CHEN; SZE, 2017). Camadas nas extremidades da rede (primeiras e últimas) tendem a ser mais sensíveis, enquanto camadas intermediárias frequentemente possuem maior redundância e toleram taxas mais agressivas. Se essa hipótese for confirmada pelos dados de sensibilidade coletados, a alocação adaptativa de taxas deve permitir atingir níveis de compressão global comparáveis aos dos métodos uniformes com menor degradação de acurácia. O processo detalhado de análise de sensibilidade e a formalização da alocação de taxas são apresentados na Seção 4.6.

Por fim, a decisão de repetir cada configuração experimental com três sementes aleatórias distintas reflete um balanço entre rigor estatístico e viabilidade computacional. Com $n = 3$ repetições, é possível calcular médias e desvios-padrão para todas as métricas, o que permite identificar se tendências observadas são consistentes ou resultado de variação de execução, especialmente nas medições de consumo energético — naturalmente mais ruidosas que as de acurácia. Reconhece-se, contudo, que esse número de repetições oferece poder estatístico limitado; comparações com diferenças próximas ao nível de ruído devem ser interpretadas com cautela. Essa limitação é discutida nas Considerações Finais do trabalho.

As seções seguintes deste capítulo detalham individualmente cada componente do desenho experimental: o *dataset* e seus conjuntos de divisão (Seção 4.2), o modelo e o protocolo de *transfer learning* (Seção 4.3), os dois métodos de poda da literatura com seus protocolos de avaliação (Seção 4.4), a estratégia otimizada por sensibilidade por camada (Seção 4.6), o *pipeline* experimental completo (Seção 4.7) e o conjunto de métricas de avaliação (Seção 4.8).

4.2 Dataset

A escolha do *dataset* condiciona, de maneira fundamental, a complexidade da tarefa de classificação, o tempo de treinamento e a relevância dos resultados perante a literatura. Em experimentos comparativos de poda neural, o conjunto de dados precisa ser suficientemente desafiador para que as diferenças entre métodos se manifestem de forma mensurável, mas operacionalmente viável o suficiente para que múltiplas repetições sejam executáveis no escopo de um trabalho de graduação. Conforme discutido na Seção 4.1, o CIFAR-100 atende a esse equilíbrio e foi selecionado como *dataset* principal deste trabalho. Esta seção detalha as características técnicas do conjunto, apresenta a justificativa acadêmica de sua escolha, descreve o pré-processamento aplicado e formaliza o protocolo de divisão dos dados.

4.2.1 O CIFAR-100

O CIFAR-100 foi introduzido por Krizhevsky (2009) no relatório técnico “Learning Multiple Layers of Features from Tiny Images”, publicado pelo Departamento de Ciência da Computação da Universidade de Toronto. O conjunto é derivado de um subconjunto anotado do projeto “80 Million Tiny Images”, uma coleção massiva de imagens de baixa resolução coletadas automaticamente da internet. A proposta de Krizhevsky (2009) foi selecionar e revisar manualmente as anotações de uma fração desse projeto, produzindo dois *datasets* com rótulos confiáveis e estrutura padronizada: o CIFAR-10 e o CIFAR-100. Desde sua publicação, ambos tornaram-se referências canônicas em avaliações de arquiteturas de redes neurais convolucionais (HE et al., 2016; KRIZHEVSKY; SUTSKEVER; HINTON, 2012) e, em particular, em estudos de poda neural (HAN et al., 2015; LI et al., 2017; FRANKLE; CARBIN, 2019; BLALOCK et al., 2020).

O CIFAR-100 contém 60.000 imagens coloridas de resolução 32×32 pixels, codificadas nos três canais RGB. Essas imagens estão distribuídas em 100 classes mutuamente exclusivas, com exatamente 600 imagens por classe. A partição oficial do conjunto reserva 500 imagens por classe para treinamento e 100 para teste, totalizando 50.000 exemplos de treinamento e 10.000 de teste. As classes cobrem uma ampla variedade de categorias de objetos do mundo real: animais (mamíferos, insetos, peixes, répteis), veículos, plantas, cenas externas e objetos domésticos, entre outras.

Um aspecto estrutural relevante do CIFAR-100 é sua organização hierárquica: as 100 classes de granularidade fina estão agrupadas em 20 superclasses de granularidade grossa. Por exemplo, as classes “leopardo”, “leão”, “tigre” e “urso” pertencem à superclasse “grandes carnívoros”; “ônibus”, “caminhão” e “trator” compõem a superclasse “veículos 2”. Essa estrutura é um diferencial em relação ao CIFAR-10 e permite que experimentos futuros avaliem tanto a classificação fina (100 classes) quanto a grossa (20 superclasses),

embora neste trabalho seja adotada exclusivamente a tarefa de classificação nas 100 classes.

Em termos de resolução, as imagens de 32×32 pixels contrastam fortemente com as 224×224 pixels tipicamente empregadas em experimentos com ImageNet. Essa diferença de resolução implica que o CIFAR-100 representa uma tarefa de classificação visual em formato compacto, condição distinta das imagens de alta resolução tipicamente empregadas em modelos pré-treinados em ImageNet. Para redes profundas como a ResNet-50 (HE et al., 2016), treinadas originalmente para imagens de maior resolução, a adaptação ao formato 32×32 requer atenção ao pré-processamento — aspecto tratado na Subseção 4.2.3.

4.2.2 Justificativa da Escolha

A seleção do CIFAR-100 como *dataset* principal é uma decisão metodológica de consequências substantivas para o escopo e a generalidade dos resultados. Esta subseção articula a defesa dessa escolha em contraposição às alternativas consideradas.

A alternativa natural de maior escala seria o ImageNet-1K, com 1,28 milhão de imagens de treinamento distribuídas em 1.000 classes e resolução padrão de 224×224 pixels. O ImageNet-1K constitui o padrão-ouro em *benchmarks* de classificação visual e é, inclusive, a fonte dos pesos pré-treinados utilizados neste trabalho. Contudo, o foco deste estudo não é reproduzir um *benchmark* de larga escala, mas analisar comparativamente o efeito de diferentes paradigmas de poda sobre acurácia e consumo energético. Essa análise exige múltiplas repetições de cada configuração experimental — quatro taxas de poda por método, dois métodos da literatura mais a estratégia otimizada, e três sementes por configuração — totalizando dezenas de ciclos completos de poda e *fine-tuning*. O custo computacional de cada ciclo sobre o ImageNet-1K seria proibitivo para o escopo deste trabalho, sem que tal custo adicional trouxesse ganho proporcional para responder à pergunta de pesquisa. Não se trata, portanto, de uma limitação de recursos que impede o uso de ImageNet-1K, mas de uma escolha deliberada de proporcionalidade: o ImageNet-1K excede o que é necessário para as comparações pretendidas.

No extremo oposto do espectro de complexidade situa-se o CIFAR-10, com apenas dez classes. Redes modernas atingem acurácias superiores a 95% em CIFAR-10, mesmo após poda agressiva, o que produz diferenças de desempenho entre métodos que mal superam o desvio-padrão das medições — tornando a comparação pouco discriminativa. O CIFAR-100, com 100 classes, opera em uma faixa de acurácia tipicamente entre 60% e 80% para modelos como a ResNet-50 após *fine-tuning*, faixa na qual a degradação causada por diferentes taxas e granularidades de poda é claramente mensurável e diferenciada entre métodos.

Uma terceira família de opções inclui *datasets* de escala intermediária, como o Tiny ImageNet (200 classes, resolução 64×64). Esses conjuntos são menos adotados na literatura específica de poda, o que comprometeria a comparabilidade direta dos resultados com trabalhos de referência. A representatividade do CIFAR-100 na literatura é, por si mesma, um critério de escolha relevante: Li et al. (2017) utilizam o CIFAR-10 e o CIFAR-100 como *datasets* de avaliação primários; Frankle e Carbin (2019) empregam o CIFAR-10 para experimentos centrais; Blalock et al. (2020), no amplo levantamento do estado da arte em poda neural, observam que CIFAR-10 e CIFAR-100 são os *datasets* mais frequentemente reportados nos trabalhos da área. Adotar o CIFAR-100 garante que os resultados obtidos possam ser situados diretamente em relação a essa base de comparação consolidada.

Do ponto de vista técnico, o CIFAR-100 reúne as propriedades necessárias para os objetivos do trabalho: complexidade adequada para revelar diferenças entre métodos de poda, tamanho operacional que viabiliza múltiplas repetições, e ampla adoção na literatura de compressão de modelos. É por mérito técnico, portanto, e não por conveniência, que o CIFAR-100 é adotado como *dataset* principal.

Reconhece-se, contudo, que essa escolha implica uma limitação de escopo: os resultados obtidos descrevem o comportamento dos métodos de poda em um regime de baixa resolução com 100 classes, e não podem ser extrapolados diretamente para cenários de larga escala como o ImageNet-1K, onde efeitos de escala podem alterar os padrões de redundância e sensibilidade das camadas. Essa limitação é reconhecida formalmente na discussão das limitações metodológicas do trabalho.

4.2.3 Pré-processamento

O pré-processamento das imagens é etapa essencial em experimentos com redes neurais convolucionais. Duas funções principais são desempenhadas pelo conjunto de transformações aplicadas antes do treinamento: a normalização dos dados de entrada, que melhora o condicionamento numérico das operações de *backpropagation* (GOODFELLOW; BENGIO; COURVILLE, 2016), e a aumento de dados (*data augmentation*), que reduz o risco de *overfitting* ao ampliar artificialmente a diversidade do conjunto de treinamento (GOODFELLOW; BENGIO; COURVILLE, 2016). O protocolo adotado neste trabalho segue as transformações estabelecidas por He et al. (2016) para experimentos com ResNet em CIFAR, posteriormente consolidadas como padrão nos trabalhos de poda que utilizam esse *dataset* (LI et al., 2017).

A normalização é realizada por canal RGB: cada canal de cada imagem tem sua média subtraída e é dividido pelo desvio-padrão correspondente, ambos calculados sobre o conjunto de treinamento do próprio CIFAR-100. Esse procedimento desloca a distribuição dos valores de entrada para uma região centrada em zero com variância

unitária, o que torna o gradiente computado pelo *backpropagation* mais bem-comportado e favorece a convergência do otimizador (GOODFELLOW; BENGIO; COURVILLE, 2016). A normalização é aplicada tanto no treinamento quanto na avaliação, pois a rede foi treinada sob essa escala de entrada e suas previsões pressupõem esse condicionamento.

A aumento de dados restringe-se à fase de treinamento e compreende duas transformações: *random crop* com *padding* de 4 pixels e *horizontal flip* aleatório. O *random crop* adiciona quatro pixels de borda à imagem (preenchidos com zeros) e recorta uma janela de 32×32 pixels em posição aleatória, simulando pequenas variações de posição e enquadramento do objeto. O *horizontal flip* espelha horizontalmente a imagem com probabilidade 0,5, introduzindo invariância à orientação lateral. Juntas, essas duas transformações constituem o protocolo canônico de aumento para CIFAR, adotado em He et al. (2016) e em praticamente toda a literatura de poda subsequente (LI et al., 2017; FRANKLE; CARBIN, 2019). Durante a avaliação, nenhuma aumento é aplicada: as imagens são convertidas para *tensor* e normalizadas, refletindo o comportamento do modelo em condições reais de inferência.

A Tabela 2 resume as transformações aplicadas a cada fase.

Tabela 2 – Transformações de pré-processamento aplicadas às imagens do CIFAR-100.

Transformação	Treinamento	Avaliação
<i>Random crop</i> (32×32 , <i>padding</i> 4)	✓	—
<i>Horizontal flip</i> aleatório ($p = 0,5$)	✓	—
Conversão para <i>tensor</i>	✓	✓
Normalização (média e desvio-padrão por canal RGB)	✓	✓

Fonte: Elaboração própria, com protocolo baseado em He et al. (2016).

4.2.4 Conjuntos de Treinamento, Validação e Teste

O protocolo de divisão dos dados segue a partição oficial do CIFAR-100, com uma modificação: dos 50.000 exemplos oficiais de treinamento, 10% são reservados para compor um conjunto de validação, totalizando 45.000 exemplos efetivos de treinamento e 5.000 de validação. O conjunto de teste oficial, com 10.000 exemplos, é mantido intacto e utilizado exclusivamente para a avaliação final dos modelos. Essa separação é metodologicamente necessária para evitar que decisões de ajuste de hiperparâmetros — como a seleção da época de melhor desempenho (*early stopping*) — sejam tomadas com base no mesmo conjunto usado para reportar os resultados finais, o que introduziria viés otimista nas métricas reportadas (GOODFELLOW; BENGIO; COURVILLE, 2016).

A divisão entre treinamento e validação é realizada de forma estratificada por classe: para cada uma das 100 classes, retira-se proporcionalmente a mesma fração de exemplos (10%), garantindo que todas as classes estejam representadas com a mesma

densidade relativa nos dois subconjuntos. Sem estratificação, a amostragem aleatória simples pode, por flutuação estatística, sub-representar ou super-representar determinadas classes no conjunto de validação, introduzindo viés na estimativa de acurácia durante o treinamento e distorcendo o critério de *early stopping*. A estratificação mitiga esse risco de forma determinística.

A divisão é fixada por semente aleatória, de modo que exatamente o mesmo subconjunto de validação é usado em todos os experimentos — independentemente do método de poda, da taxa ou da semente usada no treinamento. Essa fixação isola o efeito do método de poda do efeito da composição do conjunto de validação, garantindo que diferenças de desempenho entre configurações não sejam atribuíveis a variações na divisão dos dados. O princípio de reprodutibilidade que orienta essa escolha é discutido em maior detalhe na seção de reprodutibilidade do trabalho.

4.2.5 Carregamento e Acesso

O acesso ao CIFAR-100 é feito por meio da classe `torchvision.datasets.CIFAR100`, componente da biblioteca `torchvision` distribuída com o PyTorch (PASZKE et al., 2019). Essa interface padroniza o carregamento, realiza o *download* automático do arquivo comprimido na primeira execução e disponibiliza os dados já em formato compatível com o *pipeline* de treinamento do PyTorch. O uso de uma API oficial evita inconsistências de formato e simplifica a reprodução dos experimentos em ambientes distintos.

O carregamento durante o treinamento e a avaliação é gerenciado pelo componente `DataLoader` do PyTorch, que percorre o conjunto de dados em lotes (*batches*) de tamanho configurável. Os hiperparâmetros do `DataLoader` — tamanho do lote, embaralhamento (*shuffle*) e número de trabalhadores paralelos de leitura (`num_workers`) — são definidos nos arquivos de configuração YAML do projeto, de modo que qualquer alteração fica rastreada no controle de versão. O embaralhamento é habilitado apenas no conjunto de treinamento, garantindo que cada época apresente os exemplos em ordem diferente sem perturbar a avaliação. Uma vantagem prática do CIFAR-100 é seu tamanho reduzido: o conjunto completo ocupa aproximadamente 170 MB descomprimido e pode ser integralmente carregado em memória RAM antes do início do treinamento, eliminando a latência de leitura de disco a cada iteração e acelerando substancialmente os experimentos em comparação com *datasets* de maior escala.

4.3 Modelo

Em um trabalho experimental de poda neural, a escolha da arquitetura não é um detalhe de implementação — ela define o “alvo” sobre o qual os métodos de compressão atuam. Diferentes arquiteturas apresentam propriedades estruturais distintas: grau de

redundância paramétrica, presença de conexões de atalho, densidade dos tensores intermediários, entre outras. Essas propriedades condicionam tanto a efetividade dos métodos de poda quanto os desafios de implementação envolvidos. Conforme discutido na Subseção 4.1.2, a arquitetura selecionada para este trabalho é a ResNet-50 (HE et al., 2016), pré-treinada em ImageNet-1K e adaptada ao CIFAR-100 por meio de *transfer learning*. Esta seção detalha a família arquitetural à qual pertence esse modelo, descreve sua estrutura interna, defende a escolha e apresenta o protocolo de adaptação ao domínio-alvo.

4.3.1 A Família de Redes Residuais (ResNet)

O desenvolvimento das redes neurais convolucionais profundas foi marcado, ao longo da primeira metade da década de 2010, por uma progressão constante em profundidade e acurácia nos *benchmarks* de visão computacional. O *ImageNet Large Scale Visual Recognition Challenge* (ILSVRC), competição anual que avalia a classificação de 1,28 milhão de imagens em 1.000 categorias, tornou-se o principal termômetro desse progresso. A AlexNet (KRIZHEVSKY; SUTSKEVER; HINTON, 2012) inaugurou a era das CNNs profundas em 2012 com oito camadas e erro top-5 de 15,3%; arquiteturas subsequentes, como VGGNet e GoogLeNet, empurram essa fronteira para 16 a 22 camadas, chegando a erros de 6 a 7%. Contudo, à medida que a profundidade aumentava, emergiu um obstáculo fundamental: simplesmente empilhar mais camadas deixava de melhorar, e frequentemente degradava, o desempenho do modelo — fenômeno documentado empiricamente e denominado problema da degradação.

A causa subjacente desse fenômeno é o desvanecimento do gradiente (*vanishing gradient*): em redes muito profundas, o sinal de gradiente propagado pelo *backpropagation* tende a se tornar exponencialmente pequeno à medida que atravessa camadas sucessivas, tornando o aprendizado das camadas iniciais ineficaz (GOODFELLOW; BENGIO; COURVILLE, 2016). Mesmo com inicializações cuidadosas e normalização por lotes (*batch normalization*), redes com dezenas de camadas apresentavam dificuldades de otimização que não podiam ser atribuídas ao sobreajuste: o erro de treinamento de redes mais profundas era, contra-intuitivamente, maior do que o de redes mais rasas.

A solução proposta por He et al. (2016) foi estrutural: em vez de aprender diretamente a transformação desejada $H(x)$, cada bloco da rede passa a aprender a *função residual* $F(x) = H(x) - x$, de modo que a saída do bloco é $F(x) + x$. Essa identidade aditiva é implementada por uma conexão de atalho (*skip connection*) que soma a entrada do bloco à sua saída, contornando as camadas intermediárias. O efeito prático é que o gradiente pode fluir diretamente pelo atalho durante o *backpropagation*, sem ser atenuado pelas transformações não lineares do bloco. Caso as camadas intermediárias aprendam a função zero, a saída do bloco simplesmente replica a entrada — comportamento de identidade que é muito mais fácil de aprender do que uma transformação arbitrária.

Os resultados obtidos com essa arquitetura foram expressivos. A ResNet com 152 camadas venceu o ILSVRC 2015 com erro top-5 de 3,57% — abaixo do desempenho humano estimado na tarefa (HE et al., 2016). Mais relevante para este trabalho, a família ResNet tornou-se rapidamente o modelo de referência da literatura de compressão de redes neurais: Han et al. (2015) utilizam variantes da ResNet em seus experimentos de poda por magnitude; Li et al. (2017) propõem a poda estruturada por norma L_1 de filtros avaliando-a especificamente em ResNets; e Blalock et al. (2020), em amplo levantamento do estado da arte em poda neural, identificam a ResNet como a arquitetura mais frequentemente reportada nos estudos da área.

4.3.2 Arquitetura ResNet-50

A ResNet-50 é a variante de 50 camadas da família ResNet. Sua estrutura é organizada em cinco estágios: uma camada convolucional inicial de 7×7 com *stride* 2 e 64 filtros, seguida de uma camada de *max pooling* com *stride* 2, e quatro estágios residuais compostos, respectivamente, por 3, 4, 6 e 3 blocos. A saída do último estágio é submetida a uma camada de *average pooling* global, produzindo um vetor de 2.048 dimensões que alimenta a camada totalmente conectada final. No total, a arquitetura possui aproximadamente 25,6 milhões de parâmetros treináveis (HE et al., 2016).

O bloco fundamental da ResNet-50 é o *bottleneck block* (bloco gargalo), que difere do bloco residual básico da ResNet-18 e da ResNet-34. Cada bloco gargalo é composto por uma sequência de três camadas convolucionais: uma convolução 1×1 que reduz a dimensionalidade de canal (por exemplo, de 256 para 64 canais), uma convolução 3×3 que processa espacialmente a representação comprimida, e uma convolução 1×1 que restaura a dimensionalidade de canal ao valor original (de volta a 256 canais). Essa estrutura é denominada “gargalo” porque a convolução 3×3 opera sobre um número muito menor de canais do que a representação de entrada, reduzindo o custo computacional do bloco de forma substancial sem comprometer a capacidade expressiva da rede como um todo.

Quando o número de canais muda entre a entrada e a saída de um bloco — o que ocorre nas transições entre estágios residuais —, a *skip connection* não pode realizar uma soma direta, pois os tensores têm dimensões incompatíveis. Nesses casos, a ResNet-50 emprega uma *projection shortcut*: uma convolução 1×1 adicional, com *stride* 2 e o número de filtros correspondente à dimensão de saída, é aplicada sobre a entrada do bloco antes de somá-la à saída das camadas convolucionais. Essa projeção garante a compatibilidade dimensional e preserva o fluxo de gradiente pela conexão de atalho.

Do ponto de vista da poda estruturada, a presença de *skip connections* e *projection shortcuts* introduz restrições de dimensionalidade não triviais. A remoção de filtros de uma camada convolucional interna ao bloco altera o número de canais de saída dessa camada, o que, por sua vez, afeta a dimensão de entrada da camada seguinte. Quando a

camada afetada está conectada, direta ou indiretamente, a uma *skip connection*, a poda deve ser propagada de forma consistente por todas as camadas correlacionadas para que as somas residuais permaneçam dimensionalmente compatíveis. Esse desafio de propagação de dependências é a principal razão pela qual a implementação da poda estruturada em ResNets requer ferramentas especializadas, como a biblioteca *Torch-Pruning* (FANG et al., 2023), descrita na Seção 4.5.

4.3.3 Justificativa da Escolha da ResNet-50

A escolha da ResNet-50 como modelo de referência é sustentada por três argumentos complementares. O primeiro é sua maturidade e representatividade na literatura de poda. Como registrado por Blalock et al. (2020) em levantamento abrangente, a ResNet é a arquitetura mais frequentemente empregada em estudos de poda neural publicados até 2020. Trabalhos seminais como os de Han et al. (2015) e Li et al. (2017) — que constituem justamente os dois métodos avaliados neste trabalho — utilizam a ResNet como alvo experimental central. Adotar a mesma arquitetura permite que os resultados obtidos sejam diretamente contextualizados em relação a essa base de comparação consolidada, sem a ambiguidade introduzida por escolhas idiossincráticas de modelo.

O segundo argumento diz respeito ao tamanho intermediário da arquitetura. Com 25,6 milhões de parâmetros e profundidade de 50 camadas, a ResNet-50 é suficientemente grande para que as diferenças de consumo energético entre configurações de poda sejam mensuráveis — modelos muito pequenos produzem reduções absolutas de energia tão baixas que se confundem com o ruído de medição. Ao mesmo tempo, o tamanho da ResNet-50 é operacionalmente viável: cada ciclo completo de poda e *fine-tuning* no CIFAR-100 pode ser executado em tempo razoável, o que é condição necessária para que as dezenas de configurações experimentais previstas (dois métodos da literatura, quatro taxas de poda, estratégia otimizada, três sementes cada) sejam completadas no escopo deste trabalho. Variantes menores, como a ResNet-18 com aproximadamente 11,7 milhões de parâmetros, foram descartadas precisamente por oferecerem menor margem de compressão e diferenças de consumo energético próximas ao limiar de detecção.

O terceiro argumento é arquitetural: a presença de *skip connections* torna a ResNet-50 um banco de testes realista para a poda estruturada, em contraste com redes sequenciais como a VGG-16, nas quais a propagação dos cortes é trivial. Ao impor restrições de dimensionalidade que devem ser respeitadas em toda a topologia do modelo, a ResNet-50 força uma implementação de poda mais robusta e generaliza melhor para cenários práticos. Arquiteturas projetadas explicitamente para eficiência — como MobileNet, EfficientNet e SqueezeNet — foram descartadas como modelo principal porque já incorporam, por projeto, mecanismos implícitos de compressão (convoluções separáveis em profundidade, gargalos invertidos, escalonamento composto), o que confunde a análise

do efeito isolado da poda sobre um modelo denso convencional. Modelos baseados em atenção (*Vision Transformers*), por sua vez, foram descartados por uma razão distinta: os métodos de poda estruturada para *transformers* são ainda recentes e não consolidaram um repertório comparável ao disponível para CNNs clássicas, o que prejudicaria a comparação direta com os trabalhos de referência adotados (HAN et al., 2015; LI et al., 2017).

4.3.4 Aprendizado por Transferência

O *transfer learning* (aprendizado por transferência) é uma estratégia de treinamento na qual um modelo desenvolvido para uma tarefa é reutilizado como ponto de partida para uma tarefa diferente (GOODFELLOW; BENGIO; COURVILLE, 2016). Em vez de inicializar os pesos da rede aleatoriamente e treiná-la do zero, parte-se de pesos já ajustados em um conjunto de dados de larga escala — no caso das CNNs de visão computacional, tipicamente o ImageNet-1K — e o modelo é então *fine-tuned* (ajustado finamente) sobre o conjunto de dados da tarefa-alvo. A ideia subjacente é que as representações visuais aprendidas em ImageNet — que capturam bordas, texturas, formas e padrões de objetos em grande variedade — são, em larga medida, transferíveis para outras tarefas de classificação visual, mesmo que as categorias-alvo sejam distintas.

A adoção do *transfer learning* neste trabalho é justificada por três razões complementares. A primeira é o custo computacional: treinar a ResNet-50 do zero em CIFAR-100 até a convergência exigiria um número substancialmente maior de épocas e, consequentemente, tempo de processamento incompatível com as dezenas de repetições experimentais previstas. Partir de pesos pré-treinados permite obter um *baseline* competitivo em um número reduzido de épocas de *fine-tuning*. A segunda razão é a qualidade das representações: pesos pré-treinados em ImageNet-1K codificam características visuais hierárquicas de alta qualidade que, ao serem transferidas, elevam o nível de desempenho inicial do modelo e possibilitam um *baseline* mais forte, tornando os efeitos da poda mais facilmente observáveis sobre uma referência de alta acurácia. A terceira razão é o realismo de cenário: na prática industrial, modelos raramente são treinados do zero; o protocolo mais comum é exatamente a adaptação de modelos pré-treinados a domínios específicos (HAN et al., 2015; LI et al., 2017). Adotar essa mesma abordagem garante que o protocolo experimental seja representativo do contexto real de aplicação.

A principal tensão introduzida pelo *transfer learning* neste cenário é a diferença de resolução entre o domínio de origem e o domínio-alvo: a ResNet-50 foi treinada com imagens de 224×224 pixels, enquanto as imagens do CIFAR-100 têm apenas 32×32 pixels. Essa discrepância afeta as camadas iniciais da rede, que foram otimizadas para extrair características em escala de resolução muito maior. A mitigação adotada para esse problema é descrita na subseção a seguir.

4.3.5 Adaptação da Cabeça de Classificação

A adaptação da ResNet-50 pré-treinada ao CIFAR-100 requer, em primeiro lugar, a substituição da camada de classificação final. Na configuração original do ImageNet-1K, a camada totalmente conectada (**fc**) projeta o vetor de características de 2.048 dimensões em 1.000 logits, correspondentes às 1.000 classes do ImageNet. Para a tarefa de classificação nas 100 classes do CIFAR-100, essa camada é substituída por uma nova camada totalmente conectada com 2.048 entradas e 100 saídas. Apenas essa camada de saída é reinicializada; todos os demais pesos da rede — das camadas convolucionais dos quatro estágios residuais à camada de *average pooling* — são preservados com os valores pré-treinados como ponto de partida para o *fine-tuning*. A nova camada **fc** é inicializada pelo esquema de *Kaiming He* (HE et al., 2016), que escala os pesos de forma proporcional ao número de entradas da camada, preservando a variância das ativações durante a propagação direta e favorecendo a convergência inicial.

O protocolo de adaptação adota *fine-tuning* completo: todos os parâmetros da rede, incluindo os das camadas convolucionais pré-treinadas, são atualizados durante o treinamento no CIFAR-100. A alternativa de *feature extraction* — na qual os pesos das camadas convolucionais são congelados e apenas a camada de classificação é treinada — foi descartada porque, na presença da discrepância de resolução entre ImageNet e CIFAR-100, as ativações produzidas pelas camadas intermediárias com pesos fixos seriam menos adequadas à nova escala de entrada; o *fine-tuning* completo permite que as camadas se adaptem progressivamente ao regime de resolução reduzida. Essa decisão é consistente com os protocolos adotados pelos trabalhos de referência em poda (HAN et al., 2015; LI et al., 2017), que partem igualmente de modelos pré-treinados com todos os parâmetros ajustáveis durante o *fine-tuning* no conjunto-alvo.

4.4 Métodos de Poda da Literatura

Este trabalho implementa e avalia dois métodos de poda consolidados na literatura, selecionados estrategicamente por sua complementaridade conceitual: um representativo da família de métodos não estruturados e outro da família de métodos estruturados. A comparação sistemática entre ambos permite analisar, para além das métricas de acurácia, o impacto prático de cada paradigma sobre o consumo energético em *hardware* convencional.

4.4.1 Método 1: Poda Não Estruturada por Magnitude

O primeiro método implementado é a poda não estruturada por magnitude, proposta originalmente por Han *et al.* (HAN et al., 2015) e consolidada como *baseline* de comparação em grande parte da literatura de compressão de modelos.

O método opera de forma granular sobre os pesos individuais das camadas convolucionais e totalmente conectadas. Após o treinamento do modelo denso, calcula-se o valor absoluto de cada peso, e os $k\%$ parâmetros de menor magnitude são fixados em zero. Os pesos remanescentes passam, então, por uma etapa de *fine-tuning* para compensar a perda de capacidade expressiva da rede.

A implementação utiliza a função `prune.global_unstructured` do PyTorch, aplicando um critério de norma $L1$. O uso de uma abordagem global — onde um limiar único é aplicado ao conjunto total de pesos podáveis do modelo — permite uma distribuição de esparsidade heterogênea e adaptativa, preservando camadas mais sensíveis e penalizando as mais redundantes, o que tipicamente supera a poda uniforme por camada (HAN et al., 2015). A camada de classificação final (`fc`) é intencionalmente excluída do processo devido à sua alta sensibilidade e importância crítica para o mapeamento das predições.

Do ponto de vista de desempenho de *hardware*, conforme discutido na Seção 2.5, este método gera matrizes esparsas por meio de mascaramento lógico, sem remover as estruturas físicas. Consequentemente, em arquiteturas de processamento convencionais, as operações de multiplicação por zero continuam sendo executadas. A inclusão deste método fundamenta-se exatamente no seu papel de controle: ele permitirá quantificar, de forma precisa, a lacuna existente entre a compressão teórica (esparsidade dos parâmetros) e o ganho real de eficiência energética e temporal.

4.4.2 Método 2: Poda Estruturada por Norma $L1$ de Filtros

O segundo método avaliado é a poda estruturada baseada na norma $L1$ dos filtros convolucionais, introduzida por Li *et al.* (LI et al., 2017). Esta abordagem é de suma importância para o foco deste trabalho, uma vez que promove a remoção física de filtros inteiros, reduzindo ativamente as dimensões dos mapas de características e, por conseguinte, o número de operações matemáticas executadas.

Para cada camada convolucional, calcula-se a norma $L1$ de cada filtro (a soma dos valores absolutos de seus respectivos pesos). Os filtros com as menores normas são classificados como menos informativos e removidos da arquitetura. Como a eliminação de um filtro altera o formato do *tensor* de saída da referida camada, a dimensão de entrada da camada subsequente exige ajuste correspondente.

Esta adequação arquitetural torna-se um desafio não trivial em redes com fluxo não linear, como a ResNet-50. A presença de *skip connections* impõe restrições rígidas de dimensionalidade: para que os mapas de características sejam somados elemento a elemento nas conexões residuais, suas dimensões de canal devem ser estritamente idênticas. Para solucionar o alinhamento dessas dimensões, a implementação recorre à biblioteca

Torch-Pruning (FANG et al., 2023), que abstrai a topologia da rede em um grafo de dependências (*DepGraph*) e propaga automaticamente os cortes estruturais necessários por todas as camadas correlacionadas.

Diferentemente da abordagem não estruturada, este método culmina em um modelo computacionalmente mais enxuto. Com a redução efetiva das Operações de Ponto Flutuante (FLOPs) e da pegada de memória, antecipa-se uma mitigação palpável no consumo energético empírico. Semelhante ao Método 1, o modelo podado é posteriormente submetido a *fine-tuning* para o resgate de sua performance preditiva.

4.4.3 Taxas de Poda Avaliadas

Para ambos os métodos, são avaliadas as seguintes taxas de poda: 20%, 40%, 60% e 80%. Esses valores cobrem a faixa de interesse prático, desde compressões moderadas até agressivas, e são consistentes com os valores reportados nos trabalhos de referência (HAN et al., 2015; LI et al., 2017; BLALOCK et al., 2020).

4.4.4 Protocolo de *Fine-tuning* Pós-poda

Após a aplicação de cada método de poda, o modelo é submetido a um procedimento de *fine-tuning* para recuperar parte da acurácia perdida. Os hiperparâmetros utilizados são apresentados na Tabela 3.

Tabela 3 – Hiperparâmetros do *fine-tuning* pós-poda.

Hiperparâmetro	Valor
Otimizador	SGD com <i>momentum</i> 0,9
<i>Learning rate</i> inicial	1×10^{-4}
<i>Schedule</i> de <i>learning rate</i>	Constante
<i>Weight decay</i> (L_2)	5×10^{-4}
<i>Batch size</i>	128
Épocas	10

Fonte: *Elaboração própria, com valores baseados em Han et al. (2015).*

A taxa de aprendizado do *fine-tuning* é, intencionalmente, duas ordens de magnitude menor do que a do treinamento do *baseline*, conforme prática estabelecida (HAN et al., 2015). O objetivo é ajustar finamente os pesos remanescentes sem desestabilizar o modelo. O número reduzido de épocas (10) reflete o caráter de recuperação dessa fase, em contraste com o treinamento completo do *baseline*.

4.5 *Framework* de Implementação

A implementação do trabalho é feita em Python, utilizando o *framework* PyTorch (PASZKE et al., 2019) para a construção, treinamento e manipulação dos modelos. O PyTorch é uma biblioteca de aprendizado de máquina de código aberto, amplamente adotada na comunidade de pesquisa, que conta com um ecossistema maduro de ferramentas auxiliares e uma vasta oferta de modelos pré-treinados por meio do pacote `torchvision`.

Uma característica do PyTorch que motivou sua escolha é o sistema de diferenciação automática (*autograd*), que constrói dinamicamente o grafo computacional e facilita a implementação do algoritmo de *backpropagation* durante o fine-tuning pós-poda. Além disso, o *framework* provê suporte nativo para aceleração de tensores em hardware gráfico (GPU) via integração com CUDA.

Para a execução métrica e a aplicação das técnicas centrais deste estudo, o trabalho faz uso das seguintes bibliotecas especializadas:

- `torch.nn.utils.prune`: API nativa do PyTorch utilizada para a implementação da poda não estruturada por magnitude.
- **Torch-Pruning** (FANG et al., 2023): biblioteca externa dedicada à poda estruturada. Diferentemente da abordagem por máscaras, esta ferramenta realiza a remoção física de filtros e canais, propagando automaticamente as dependências estruturais entre as camadas adjacentes por meio de um grafo de dependências (DepGraph).
- **thop**: biblioteca empregada para a contagem rigorosa de operações de ponto flutuante (FLOPs), permitindo mensurar a complexidade computacional teórica do modelo a partir de um tensor de entrada representativo.
- **CodeCarbon** (COURTY et al., 2023): ferramenta de telemetria utilizada para a medição empírica do consumo de energia e estimativa das emissões de CO_2 durante a execução dos algoritmos. A estimativa baseia-se no monitoramento direto da infraestrutura de hardware (CPU/GPU) combinada à intensidade de carbono da rede elétrica correspondente à localização geográfica da execução.

4.6 Estratégia Otimizada: Sensibilidade por Camada

Os métodos descritos na Seção 4.4 aplicam uma taxa de poda global, isto é, uniforme entre camadas. No entanto, camadas diferentes de uma mesma rede podem tolerar diferentes graus de compressão: algumas podem ser podadas agressivamente sem comprometer a acurácia, enquanto outras são críticas e uma pequena poda já degrada o desempenho (HAN et al., 2015; YANG; CHEN; SZE, 2017).

Com base nessa observação, este trabalho propõe como estratégia otimizada uma poda estruturada por filtros na qual a taxa de poda aplicada a cada camada é determinada a partir de uma análise prévia de sensibilidade. O processo é composto por duas etapas principais.

4.6.1 Etapa 1 – Análise de Sensibilidade por Camada

Partindo do modelo *baseline* já treinado, cada camada convolucional é podada individualmente, em isolamento, em diferentes taxas (10%, 20%, ..., 90%). Para cada combinação (camada, taxa), a acurácia do modelo é medida *sem* realizar *fine-tuning*, a fim de capturar a sensibilidade intrínseca daquela camada à remoção de filtros. O resultado é um perfil de sensibilidade por camada, que pode ser representado, para cada camada ℓ , como uma função $s_\ell(p) = \text{acurácia}(\text{modelo com camada } \ell \text{ podada em taxa } p)$.

4.6.2 Etapa 2 – Alocação Adaptativa de Taxas

A partir do perfil de sensibilidade obtido na Etapa 1, define-se, para cada camada, uma taxa específica de poda, respeitando um orçamento global de compressão (por exemplo, redução total de 50% dos filtros). Camadas mais robustas, isto é, que sofreram menor queda de acurácia ao serem podadas isoladamente, recebem taxas maiores; camadas mais sensíveis recebem taxas menores. A formalização exata da alocação será apresentada e validada experimentalmente.

Em seguida, aplica-se a poda estruturada L_1 com essas taxas individuais, e o modelo resultante é submetido a *fine-tuning* com os mesmos hiperparâmetros descritos na Tabela 3.

A expectativa é que essa estratégia otimizada permita atingir taxas globais de compressão próximas às dos métodos uniformes, porém com menor perda de acurácia, em razão da alocação não-uniforme do “orçamento” de poda.

4.7 Pipeline Experimental

Para cada combinação (método de poda, taxa de poda, repetição), o procedimento experimental adotado é o seguinte:

1. **Carregamento do *baseline*:** o modelo ResNet-50 previamente treinado no CIFAR-100 é carregado a partir de um *checkpoint* salvo em disco.
2. **Aplicação da poda:** o método selecionado é aplicado com a taxa configurada.
3. ***Fine-tuning*:** o modelo podado é retreinado conforme o protocolo da Tabela 3.

4. **Finalização do modelo:** a máscara de poda é fundida aos pesos (no caso da poda não estruturada) ou a arquitetura é consolidada (no caso da poda estruturada), e o modelo final é salvo em disco.
5. **Avaliação completa:** calculam-se todas as métricas definidas na Seção 4.8, incluindo a medição de consumo energético via **CodeCarbon** ao longo da inferência sobre o conjunto de teste.
6. **Registro dos resultados:** métricas e metadados do experimento são salvos em arquivos CSV e JSON, permitindo análise posterior.

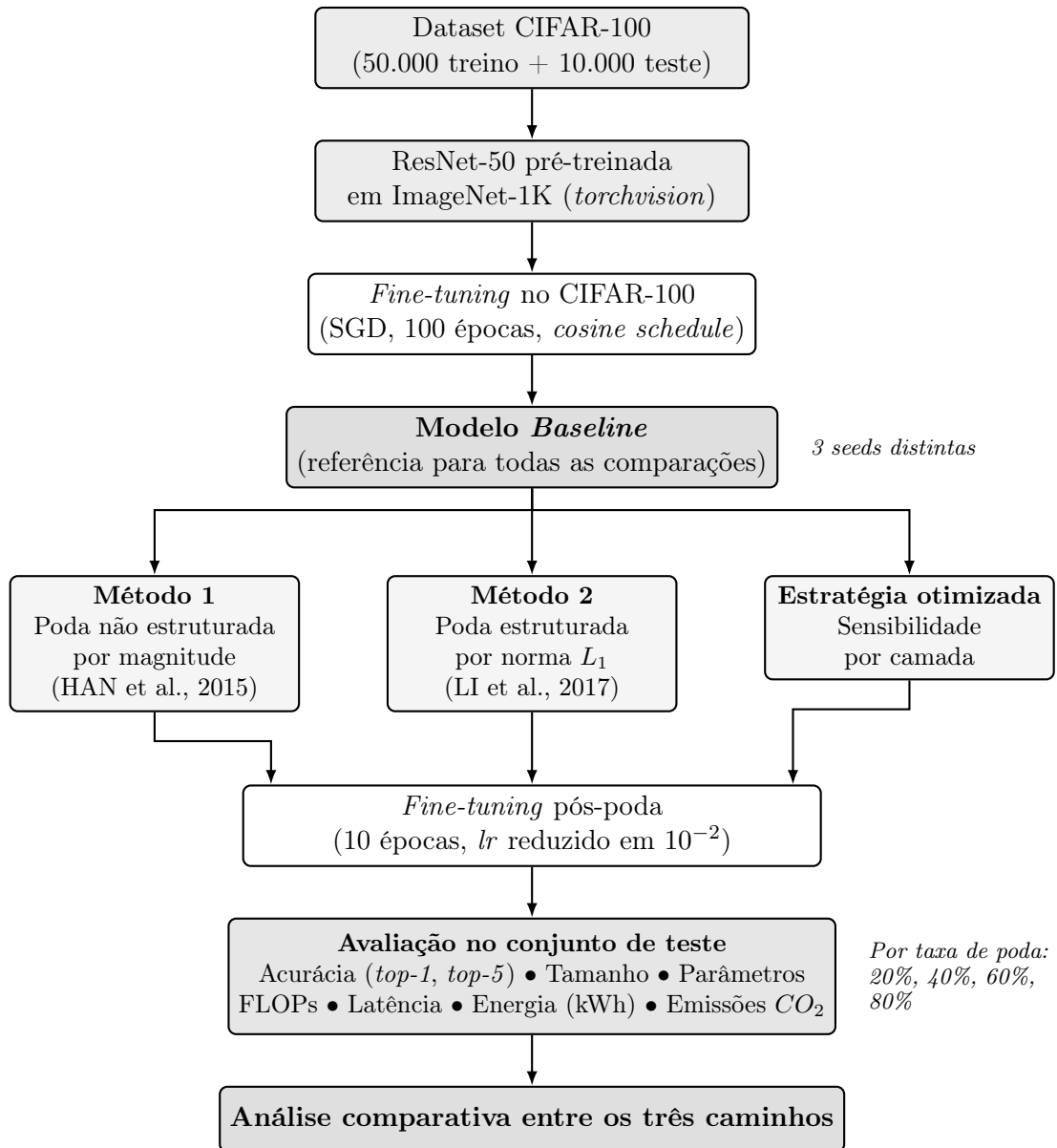
Cada combinação (método, taxa) é executada **três vezes**, com sementes aleatórias distintas, para permitir o cálculo de médias e desvios-padrão das medidas, sobretudo as de consumo energético, mais suscetíveis a variações de execução.

O fluxo geral dos experimentos é apresentado na Figura 7. A partir do *dataset* CIFAR-100 e da ResNet-50 pré-treinada, é obtido o modelo *baseline*, sobre o qual são aplicados, em paralelo, os dois métodos de poda da literatura e a estratégia otimizada. Cada um dos modelos resultantes passa por *fine-tuning* e é avaliado segundo o mesmo conjunto de métricas, permitindo comparação direta dos três caminhos.

4.8 Métricas de Avaliação

Cada modelo — *baseline*, podado pelos métodos da literatura e podado pela estratégia otimizada — é avaliado por meio de um conjunto de métricas que cobrem três dimensões: desempenho preditivo, custo computacional e impacto ambiental. A Tabela 4 resume todas as métricas, suas unidades e ferramentas.

Figura 7 – Pipeline experimental do trabalho.



Fonte: Elaboração própria.

Tabela 4 – Métricas de avaliação adotadas.

Métrica	O que mede	Unidade	Ferramenta
Acurácia (<i>top-1</i> , <i>top-5</i>)	Proporção de predições corretas	%	PyTorch
Tamanho do modelo	Espaço em disco do <code>state_dict</code>	MB	PyTorch
Parâmetros totais	Pesos treináveis	unidades	PyTorch
Parâmetros não-zero	Pesos efetivamente não nulos	unidades	PyTorch
Esparsidade	Proporção de pesos iguais a zero	%	PyTorch
FLOPs	Operações de ponto flutuante por inferência	FLOPs	<code>thop</code>
Latência	Tempo médio de uma inferência	ms	<code>time.perf_counter</code>
Energia consumida	Energia total durante a inferência	kWh	CodeCarbon
Emissões de CO_2	Emissões equivalentes ao consumo	kg	CodeCarbon

Fonte: Elaboração própria.

5 Resultados

Este capítulo apresenta os resultados experimentais obtidos com o GPT-2 *small* sobre o WikiText-2. A exposição segue a ordem em que os experimentos foram conduzidos: primeiro as duas estratégias clássicas de poda — magnitude (Seção 5.2) e estruturada (Seção 5.3) —, cujos resultados delimitam o problema; em seguida a estratégia otimizada proposta, em suas duas fases: o levantamento do perfil de sensibilidade por camada (Seção 5.4) e a varredura da alocação dele derivada (Seção 5.5). A Seção 5.6 sintetiza os achados.

5.1 Protocolo e Modelo de Referência

Todos os experimentos partem do GPT-2 *small* pré-treinado (124.439.808 parâmetros), avaliado no conjunto de teste do WikiText-2 por janela deslizante com *stride* de 512 *tokens*. A perplexity do modelo denso é de **25,17**, valor coerente com o reportado na literatura para essa configuração, o que valida a implementação do procedimento de avaliação. As medições de custo do modelo denso são: 474,70 MB de memória de parâmetros e 126,6 GFLOPs por janela de 512 *tokens*.

Duas ressalvas metodológicas condicionam a leitura das tabelas que se seguem.

A primeira é que **todos os resultados deste capítulo são *one-shot***: poda-se o modelo pré-treinado e mede-se imediatamente, sem qualquer *fine-tuning* de recuperação. Essa escolha isola o efeito da poda do efeito do retreinamento, mas implica que as perplexities aqui reportadas são substancialmente piores que as da literatura, em que a poda é seguida de retreinamento. As curvas não devem, portanto, ser lidas como proposta de configuração de uso, e sim como medida do dano que cada estratégia de alocação causa por si só.

A segunda é que as varreduras foram executadas em sessões distintas do ambiente Kaggle, e as medições de **custo dependem da sessão**. O mesmo modelo denso registrou 40,5 ms por janela em uma sessão, 54,0 ms em outra e 46,3 ms em uma terceira; o consumo de energia do modelo denso variou de forma análoga, entre 1176 e 1572 μWh . Por essa razão, comparações de tempo e energia **entre** varreduras são sempre apresentadas em termos relativos ao *baseline* da própria sessão, nunca em valores absolutos. As emissões de CO₂, que dependem adicionalmente do fator de carbono da rede elétrica do *datacenter* — que também variou entre sessões —, não são comparadas entre varreduras.

5.2 Poda por Magnitude

A Tabela 5 apresenta a varredura da poda por magnitude (HAN et al., 2015) nos dois escopos de alocação implementados: *global*, em que um único limiar sobre o valor absoluto dos pesos é aplicado a toda a rede, e *uniforme*, em que cada camada recebe a mesma fração de poda, com limiar próprio.

Tabela 5 – Poda por magnitude *one-shot* no GPT-2 *small* / WikiText-2. As colunas de custo referem-se ao escopo uniforme; o escopo global produz valores idênticos, pois nenhum dos dois altera a arquitetura.

Esparsidade	Perplexity		Custo		
	Global	Uniforme	Params. $\neq 0$	Tamanho	FLOPs
0 %	25,17	25,17	124,4 M	474,7 MB	126,6 G
10 %	25,40	25,47	115,9 M	474,7 MB	126,6 G
20 %	26,87	27,22	107,5 M	474,7 MB	126,6 G
30 %	40,23	36,42	99,0 M	474,7 MB	126,6 G
40 %	165,20	104,82	90,5 M	474,7 MB	126,6 G
50 %	1096,50	473,28	82,0 M	474,7 MB	126,6 G
60 %	5121,25	2004,20	73,5 M	474,7 MB	126,6 G
70 %	9371,16	6891,85	65,0 M	474,7 MB	126,6 G
80 %	8847,27	10871,00	56,5 M	474,7 MB	126,6 G
90 %	4612,69	13996,88	48,0 M	474,7 MB	126,6 G

O resultado mais importante da tabela está nas três últimas colunas, e é negativo: **o custo não cai**. Entre 0 % e 90 % de esparsidade, o número de parâmetros não nulos cai de 124,4 M para 48,0 M — uma redução de 61 % —, enquanto o tamanho em memória, os FLOPs e o tempo de inferência permanecem rigorosamente constantes. A explicação é direta: a poda não estruturada substitui pesos por zeros, mas não remove nada da arquitetura. Os tensores mantêm as mesmas dimensões, e a GPU multiplica os zeros exatamente como multiplicava os valores originais. O ganho de 61 % em parâmetros é contábil, não físico.

Esse achado responde empiricamente, e pela negativa, à primeira das duas perguntas que orientam este trabalho — a de se a redução teórica de parâmetros se converte em redução real de consumo. Na Tesla T4 utilizada, e sem suporte de *hardware* ou de *kernels* para esparsidade, a poda não estruturada não entrega ganho de eficiência algum. A pequena tendência de queda na energia medida (de 1572 para 1432 μWh) é da ordem da variação observada entre execuções repetidas do mesmo modelo denso e não deve ser interpretada como efeito da poda.

Quanto à qualidade, a curva de perplexity apresenta um joelho acentuado em torno de 30 %. Até 20 % de esparsidade a poda é praticamente gratuita (acréscimo de 0,3

a 2,1 pontos de perplexity); a partir de 40 % a degradação torna-se explosiva. A região utilizável em regime *one-shot* é, portanto, limitada a esparsidades de até 20 %.

Por fim, a comparação entre os dois escopos revela um cruzamento duplo: os escopos empatam até 20 %, o uniforme domina claramente entre 30 % e 70 % — exatamente a faixa em que a escolha de alocação teria consequência prática — e o global só apresenta valores menores em 80 % e 90 %, região em que ambos os modelos já estão destruídos (perplexity acima de 4600) e a comparação perde sentido. Na faixa que importa, portanto, a alocação uniforme é preferível à global. Esse é um resultado contraintuitivo: o escopo global é o que aloca o orçamento com base em uma medida de importância — a magnitude do peso — e ainda assim perde para a alocação que ignora qualquer medida de importância. Trata-se do primeiro indício de que a magnitude do peso é um *proxy* pobre para a importância de um parâmetro.

5.3 Poda Estruturada

A Tabela 6 apresenta a varredura da poda estruturada (LI et al., 2017), adaptada ao *Transformer*: remoção física de cabeças de atenção e de neurônios das camadas MLP, ordenados por norma L_1 . A esparsidade-alvo é expressa como fração de *estruturas* removidas; a fração de parâmetros efetivamente eliminada aparece como esparsidade real.

Tabela 6 – Poda estruturada *one-shot* (cabeças e neurônios MLP, escopo global) no GPT-2 *small* / WikiText-2. O tempo de inferência é normalizado pelo *baseline* da mesma sessão (40,5 ms).

Estrut.	Perplexity	Params.	Tamanho	FLOPs	Tempo rel.	Energia rel.
0 %	25,17	124,4 M	474,7 MB	126,6 G	1,000	1,000
10 %	356,20	116,0 M	442,6 MB	118,0 G	0,968	0,975
20 %	2102,06	107,6 M	410,5 MB	109,3 G	0,908	0,945
30 %	2413,52	99,0 M	377,6 MB	100,5 G	0,831	0,865
40 %	2434,92	90,6 M	345,5 MB	91,9 G	0,756	0,786
50 %	2402,58	81,9 M	312,6 MB	83,1 G	0,688	0,714
60 %	2957,26	73,5 M	280,5 MB	74,5 G	0,628	0,638
70 %	4635,36	65,1 M	248,3 MB	65,9 G	0,557	0,558
80 %	7620,02	56,5 M	215,5 MB	57,0 G	0,503	0,489
90 %	5822,89	48,1 M	183,3 MB	48,4 G	0,445	0,413

O contraste com a Tabela 5 é o achado central deste trabalho no plano da eficiência. Aqui **o custo cai, e cai proporcionalmente aos parâmetros**: a 90 % de estruturas removidas, o tempo de inferência cai 56 %, a energia 59 %, o tamanho em memória 61 % e os FLOPs 62 %, acompanhando a redução de 61 % no número de parâmetros. As duas tabelas formam, juntas, a resposta completa à primeira pergunta empírica:

a redução de parâmetros só se converte em redução de consumo quando a poda remove estruturas inteiras da arquitetura, e não quando apenas as zera.

O preço dessa eficiência, contudo, é severo em regime *one-shot*. Já com 10 % de estruturas removidas a perplexity salta de 25,17 para 356,20 — compare-se com 25,40 na poda por magnitude ao mesmo nível. **Não existe região gratuita** na poda estruturada sem retreinamento, o que é coerente com o procedimento original de Li et al. (2017), em que a poda é sempre seguida de *fine-tuning*.

Dois resultados secundários merecem registro. O primeiro é que a inversão de escopo observada na magnitude se repete ao contrário: na poda estruturada é o escopo **global** que domina, e a alocação uniforme colapsa em taxas altas (perplexity de 53.313 a 80 % e de 827.316 a 90 %, contra 7620 e 5823 do global). Somado ao achado da seção anterior, isso estabelece que **nenhuma das duas alocações ingênuas vence nos dois regimes** — a constatação que motiva diretamente a estratégia otimizada.

O segundo é a assimetria entre os dois tipos de estrutura. Remover cabeças de atenção é notavelmente mais tolerável que remover neurônios MLP: com 129 das 144 cabeças removidas (90 %), a perplexity fica em 781, ao passo que a remoção equivalente de neurônios MLP produz valores acima de 3900. A curva das cabeças é, ainda, não monotônica, com mínimo local em torno de 30 % (perplexity 162, inferior aos 237 obtidos com apenas 10 % removidas) — eco direto do achado de Michel, Levy e Neubig (2019) sobre a redundância das cabeças de atenção. Em contrapartida, as camadas MLP concentram cerca de dois terços dos parâmetros podáveis, de modo que a poda restrita a cabeças tem alcance limitado como estratégia de compressão.

5.4 Perfil de Sensibilidade por Camada

As duas seções anteriores mostraram que nenhuma alocação ingênua do orçamento de poda domina o espectro de esparsidade: na poda por magnitude o escopo uniforme prevaleceu na faixa útil (Seção 5.2), enquanto na poda estruturada foi o escopo global que obteve as melhores perplexidades (Seção 5.3) — e ambas as alocações colapsam em algum regime. A hipótese central deste trabalho é que a causa é comum aos dois casos: as camadas do *Transformer* não são igualmente importantes, de modo que tanto tratá-las de forma idêntica (alocação uniforme) quanto confiar exclusivamente na magnitude dos pesos como proxy de importância (alocação global) distribuem mal o orçamento disponível.

Esta seção reporta o experimento que testa diretamente essa hipótese. Para cada uma das 12 camadas do GPT-2 *small* e para cada uma das duas estratégias, poda-se **apenas** aquela camada a uma taxa-sonda fixa de 50 %, mantendo-se as demais intactas, e mede-se a perplexity resultante no conjunto de teste do WikiText-2. Não há *fine-tuning*

— o procedimento é *one-shot*, como nas varreduras anteriores. A degradação é registrada em escala logarítmica,

$$\Delta_\ell = \ln\left(\frac{\text{PPL}_\ell}{\text{PPL}_{\text{base}}}\right), \quad (5.1)$$

onde PPL_ℓ é a perplexity do modelo com apenas a camada ℓ podada e $\text{PPL}_{\text{base}} = 25,17$ é a perplexity do modelo denso. O logaritmo é necessário porque a perplexity de modelos podados sem recuperação varia por ordens de grandeza: em escala linear, uma única camada catastrófica tornaria as demais indistinguíveis entre si. O experimento totaliza 24 medições (12 camadas $\times$ 2 estratégias).

5.4.1 Perfil Obtido

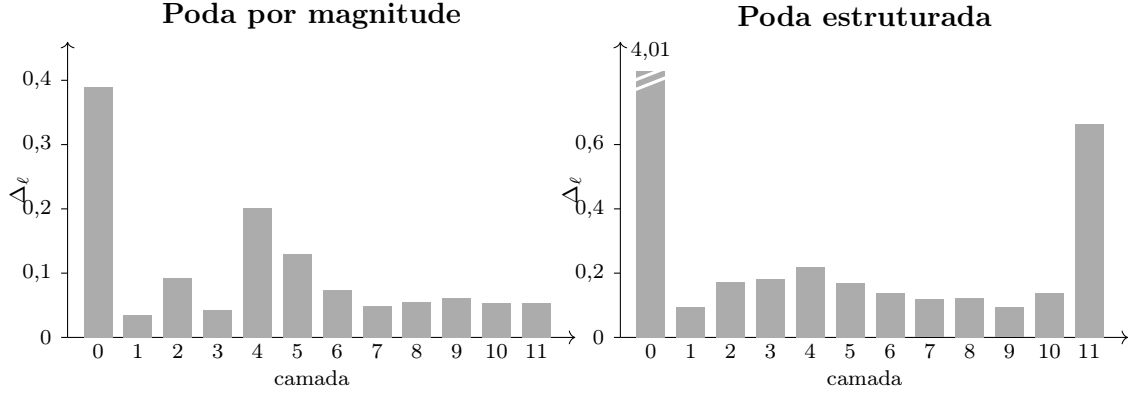
A Tabela 7 apresenta os resultados e a Figura 8 os representa graficamente.

Tabela 7 – Perfil de sensibilidade por camada do GPT-2 *small* no WikiText-2. Cada linha corresponde a podar exclusivamente a camada indicada, à taxa-sonda de 50 %, sem *fine-tuning*. Perplexity do modelo denso: 25,17.

Camada	Poda por magnitude		Poda estruturada	
	Perplexity	Δ_ℓ	Perplexity	Δ_ℓ
0	37,17	0,390	1386,43	4,009
1	26,06	0,035	27,66	0,094
2	27,62	0,093	29,91	0,173
3	26,28	0,043	30,23	0,183
4	30,81	0,202	31,31	0,218
5	28,66	0,130	29,80	0,169
6	27,09	0,074	28,90	0,138
7	26,44	0,049	28,31	0,118
8	26,59	0,055	28,45	0,122
9	26,76	0,061	27,63	0,093
10	26,55	0,054	28,89	0,138
11	26,55	0,054	48,87	0,663
Razão máx./mín.		11,3 $\times$	42,9 $\times$	

O primeiro resultado é que a hipótese se confirma de forma inequívoca: a sensibilidade está longe de ser uniforme. Sob poda por magnitude, a camada mais frágil degrada o modelo 11,3 vezes mais que a mais robusta; sob poda estruturada, 42,9 vezes mais. Tratar as 12 camadas como intercambiáveis — que é exatamente o que a alocação uniforme faz — ignora uma diferença de uma ordem de grandeza no custo de podar cada uma delas.

Figura 8 – Degradação Δ_ℓ causada pela poda isolada de cada camada, à taxa-sonda de 50 %. Note-se que as escalas verticais diferem entre os dois gráficos; na poda estruturada, a barra da camada 0 excede o eixo e está truncada.



Fonte: Elaborada pelo autor.

5.4.2 A Camada 0 como Gargalo

O segundo resultado é a magnitude do papel da primeira camada. Nas duas estratégias ela é a mais sensível, mas o efeito é qualitativamente distinto no caso estruturado: remover metade das cabeças de atenção e dos neurônios MLP **apenas do bloco 0** eleva a perplexity de 25,17 para 1386,43, ou seja, 55 vezes o valor do modelo denso. Para dimensionar esse número, a varredura estruturada uniforme a 50 % — que poda *todas* as 12 camadas — atinge perplexity 4573,00. Em escala logarítmica, a camada 0 sozinha responde por $4,01/5,20 \approx 77\%$ de toda a degradação produzida pela poda uniforme do modelo inteiro.

A interpretação é coerente com o papel funcional do primeiro bloco. É nele que as representações deixam de ser a soma direta de *embeddings* de token e de posição e passam a ser contextualizadas pela primeira vez; qualquer informação descartada ali não tem como ser recuperada pelas camadas subsequentes, que operam sobre um sinal já degradado. Trata-se, portanto, de um erro que se propaga por toda a pilha, ao contrário de um erro cometido nas camadas intermediárias.

Na poda estruturada observa-se ainda um segundo ponto crítico na outra extremidade: a camada 11, imediatamente anterior à cabeça de linguagem, apresenta $\Delta_{11} = 0,663$, quase cinco vezes a mediana das camadas intermediárias. O perfil estruturado tem, assim, formato aproximado de “U”: as bordas da pilha são críticas e o miolo é comparativamente barato. Esse padrão não se repete na poda por magnitude, cujo perfil decai após um pico inicial e mantém as últimas camadas entre as mais robustas.

5.4.3 Os Perfis Dependem da Estratégia

O terceiro resultado tem consequência metodológica direta: os dois perfis **não** são o mesmo. A correlação de postos de Spearman entre eles é $\rho = 0,50$ — há concordância, mas parcial. As duas estratégias coincidem em eleger a camada 0 como a mais crítica e em considerar as camadas 1 e 9 entre as mais robustas, e divergem em praticamente todo o resto: a camada 11 é a segunda mais sensível sob poda estruturada e apenas a oitava sob poda por magnitude; a camada 3 é a segunda mais robusta sob magnitude e a quarta mais sensível sob poda estruturada.

A implicação prática é que não existe um perfil de sensibilidade único do GPT-2. Sensibilidade é uma propriedade do par (camada, estratégia de poda), não da camada isoladamente, e o perfil precisa ser levantado separadamente para cada estratégia — como foi feito aqui — sob pena de se alocar o orçamento com um mapa que pertence a outro método. Esse achado também oferece uma explicação para a inversão observada nas varreduras ingênuas: como magnitude de peso e norma de estrutura respondem a aspectos diferentes da importância de uma camada, é esperado que o escopo global (que decide com base nessas grandezas) acerte em um caso e erre no outro.

5.4.4 Da Sensibilidade à Alocação

O perfil é um diagnóstico; para virar estratégia, precisa ser convertido em uma taxa de poda por camada. A regra adotada atribui a cada camada uma taxa proporcional à sua robustez — o inverso da degradação medida —, elevada a um expoente β e escalada por um fator único α tal que a média das taxas, ponderada pelo número de parâmetros de cada camada, iguale o orçamento global s pretendido:

$$r_\ell = \min\left(\alpha \cdot \Delta_\ell^{-\beta}, r_{\max}\right), \quad \frac{\sum_\ell w_\ell r_\ell}{\sum_\ell w_\ell} = s, \quad (5.2)$$

com w_ℓ o número de parâmetros podáveis da camada ℓ e $r_{\max} = 0,95$ um teto que impede que mesmo a camada mais robusta seja integralmente zerada. O fator α é obtido por bisseção, uma vez que a fração podada é monótona em α . O expoente β controla o quanto o perfil é levado a sério: com $\beta = 0$ a regra degenera na poda uniforme e com $\beta = 1$ usa-se o inverso puro da degradação; valores intermediários amortecem a razão entre camadas, o que importa porque essa razão é ilimitada — uma camada com Δ_ℓ muito próximo de zero receberia praticamente todo o orçamento.

A Tabela 8 mostra as taxas que a Equação 5.2 produziu para um orçamento global de 50% com $\beta = 1$. O contraste com a alocação uniforme é acentuado: sob poda estruturada, a camada 0 recebe 2% de poda e a camada 1 recebe 84%, enquanto a alocação uniforme aplicaria 50% a ambas.

Tabela 8 – Taxas de poda por camada efetivamente aplicadas pela Equação 5.2 para orçamento global de 50 % e $\beta = 1$ (coluna `taxas_por_camada` dos registros experimentais). A alocação uniforme corresponderia a 50 % em todas as camadas.

Camada	0	1	2	3	4	5	6	7	8	9	10	11
Magnitude (%)	8	94	35	75	16	25	44	67	59	53	61	61
Estruturada (%)	2	84	46	43	36	47	57	67	64	85	57	12

5.4.5 Limitações do Perfilamento

Três limitações do procedimento devem ser explicitadas, pois condicionam a leitura dos resultados da varredura otimizada.

A primeira é que a sonda foi aplicada a uma única taxa (50 %). O perfil mede, portanto, a sensibilidade das camadas naquele regime específico, e nada garante que a ordenação se mantenha em taxas muito menores ou muito maiores. A escolha por uma única sonda é uma concessão a custo: cada ponto do perfil exige uma avaliação completa de perplexity, e o levantamento com múltiplas sondas multiplicaria proporcionalmente o custo da fase.

A segunda, e mais relevante, é que a sonda mede o efeito de cada camada **isoladamente**, ignorando as interações entre elas. Os dados do próprio experimento mostram que essa aproximação é imperfeita: a soma dos Δ_ℓ individuais sob poda por magnitude é 1,238, ao passo que a poda uniforme simultânea de todas as camadas a 50 % produz degradação logarítmica de 2,934 — mais que o dobro. A degradação é fortemente superaditiva, isto é, os erros introduzidos em camadas distintas se compõem em vez de simplesmente se somarem. O perfil deve, por isso, ser lido como uma ordenação relativa de importância, e não como um modelo preditivo da degradação de uma configuração completa.

A terceira é que todas as medições são *one-shot*, sem *fine-tuning* de recuperação. Camadas que parecem frágeis nesse regime poderiam ser recuperáveis com retreinamento, o que eventualmente alteraria a alocação ótima. A opção pelo regime *one-shot* é deliberada e coerente com o objetivo do trabalho — isolar o efeito da poda do efeito do retreinamento —, mas restringe a generalidade das conclusões.

Nenhuma dessas limitações invalida o achado principal desta seção, que é robusto às três: sob qualquer leitura, as camadas do GPT-2 diferem entre si por mais de uma ordem de grandeza no custo de serem podadas, e essa diferença é informação que as alocações uniforme e global simplesmente não utilizam. Se essa informação se converte em ganho efetivo na fronteira entre compressão e qualidade é o que a varredura da estratégia otimizada, reportada na seção seguinte, se propõe a verificar.

5.5 Varredura da Alocação Guiada por Sensibilidade

Esta seção reporta a fase que responde à pergunta central do trabalho: a alocação derivada do perfil de sensibilidade melhora, de fato, o compromisso entre compressão e qualidade em relação às alocações ingênuas? As taxas por camada produzidas pela Equação 5.2 foram aplicadas nos mesmos nove níveis de orçamento das varreduras anteriores, para as duas estratégias, e avaliadas com o mesmo vetor de métricas. O experimento foi replicado com dois valores do expoente β — 1,0 e 0,5 —, o que fornece a ablação da função de alocação sobre um mesmo perfil.

5.5.1 Poda por Magnitude

A Tabela 9 compara as quatro alocações.

Tabela 9 – Perplexity da poda por magnitude sob as quatro alocações. Em negrito, o menor valor de cada linha.

Esparsidade	Alocação ingênua		Guiada por sensibilidade	
	Global	Uniforme	$\beta = 1,0$	$\beta = 0,5$
10 %	25,40	25,47	25,44	25,44
20 %	26,87	27,22	26,79	26,57
30 %	40,23	36,42	34,62	31,20
40 %	165,20	104,82	193,83	57,30
50 %	1096,50	473,28	1026,09	447,57
60 %	5121,25	2004,20	1524,55	2171,79
70 %	9371,16	6891,85	1853,30	2805,39
80 %	8847,27	10871,00	2747,56	3715,68
90 %	4612,69	13996,88	10398,10	7616,63

A alocação guiada vence em oito dos nove níveis, e as margens são substanciais na faixa de interesse. Em 30 % de esparsidade, a perplexity cai de 36,42 (melhor alocação ingênua) para 31,20 — uma redução de 14 %. Em 40 %, o ganho é de 104,82 para 57,30, ou seja, a degradação é praticamente **reduzida à metade** usando exatamente o mesmo orçamento de poda. Em 50 % a vantagem persiste, ainda que modesta (447,57 contra 473,28). Nos níveis altos, de 60 % a 80 %, a configuração com $\beta = 1$ produz perplexities entre 2,5 e 4 vezes menores que a melhor alocação ingênua — embora, nesse regime, todos os modelos já estejam degradados a ponto de a comparação ter interesse mais teórico que prático.

A única derrota ocorre em 90 %, onde o escopo global apresenta o menor valor. Como discutido na Seção 5.2, essa é a região em que a perplexity de todas as configu-

rações ultrapassa 4600 e a ordenação entre modelos igualmente inservíveis não carrega significado.

5.5.2 Poda Estruturada

Na poda estruturada, os ganhos são de outra ordem de grandeza, como mostra a Tabela 10.

Tabela 10 – Perplexity da poda estruturada sob as quatro alocações. Em negrito, o menor valor de cada linha.

Estruturas	Alocação ingênua		Guiada por sensibilidade	
	Global	Uniforme	$\beta = 1,0$	$\beta = 0,5$
10 %	356,20	481,13	124,29	215,83
20 %	2102,06	1484,89	371,07	717,19
30 %	2413,52	5467,56	724,53	923,65
40 %	2434,92	10486,71	1301,15	9489,87
50 %	2402,58	4573,00	2327,59	7922,81
60 %	2957,26	5799,76	5393,15	12322,05
70 %	4635,36	12249,58	10130,33	103463,60
80 %	7620,02	53313,38	8891,77	203995,84
90 %	5822,89	827316,19	5353,85	4830,24

Na faixa de 10 % a 50 % de estruturas removidas, a alocação guiada com $\beta = 1$ domina de forma inequívoca. Em 10 %, a perplexity cai de 356,20 para 124,29 — uma redução de 65 %. Em 20 %, de 1484,89 para 371,07, um fator de **quatro**. Em 30 %, de 2413,52 para 724,53, um fator de **3,3**. Considerando que a Seção 5.3 concluiu que não existe região gratuita para a poda estruturada *one-shot*, reduzir a perplexity a um quarto no mesmo orçamento é o resultado mais expressivo deste capítulo.

Uma ressalva quantitativa é necessária. Como a remoção de estruturas é discreta, as taxas fracionárias por camada não são atingíveis exatamente, e a alocação guiada acaba removendo uma fração de parâmetros ligeiramente menor que a ingênua no mesmo alvo: em 10 %, esparsidade real de 9,20 % contra 9,90 % do escopo global; em 30 %, 29,01 % contra 29,95 %. Parte da vantagem observada deve-se, portanto, a um corte marginalmente menor. A diferença é de menos de um ponto percentual de esparsidade, ao passo que a diferença de perplexity chega a um fator de quatro, de modo que a explicação não se sustenta como causa principal — mas o efeito existe e deve ser declarado.

A partir de 60 %, a ordenação se inverte e o escopo global volta a vencer. A causa é identificável nas taxas alocadas: a partir desse orçamento, um número crescente de camadas satura no teto $r_{\max} = 0,95$ — duas camadas em 60 %, quatro em 70 %, oito em 80 % e onze em 90 %. Saturadas as camadas, a alocação deixa de discriminar entre elas

e degenera em algo próximo da poda uniforme sobre o subconjunto saturado, ao mesmo tempo em que concentra uma pressão crescente sobre as poucas camadas ainda livres. A estratégia guiada, portanto, **só é informativa enquanto o orçamento couber dentro do teto por camada**; acima disso, ela perde sua vantagem estrutural. Esse é um limite intrínseco de qualquer alocação proporcional com teto, e não um defeito de implementação.

5.5.3 Ablação do Expoente β

A comparação entre $\beta = 1,0$ e $\beta = 0,5$ produz um resultado que não era antecipado: **o valor adequado de β depende da estratégia**, e depende dela de forma oposta nas duas.

Na poda estruturada, $\beta = 1$ domina em todos os níveis até 80 %, e o amortecimento é francamente prejudicial — em 70 % e 80 % a configuração com $\beta = 0,5$ produz perplexities de 103.463 e 203.996, uma ordem de grandeza pior que qualquer outra alocação testada. Na poda por magnitude ocorre o inverso na faixa útil: $\beta = 0,5$ é melhor de 20 % a 50 %, e só a partir de 60 % o expoente pleno passa à frente.

A explicação mais plausível liga-se à dispersão dos perfis. O perfil estruturado tem razão máximo/mínimo de $42,9\times$ e é dominado pela camada 0, cuja fragilidade é tão extrema (Seção 5.4.2) que qualquer amortecimento da alocação lhe devolve orçamento de poda com consequências catastróficas: em 70 %, $\beta = 0,5$ atribui 14,6 % de poda à camada 0, contra 3,0 % de $\beta = 1$. Quando o perfil contém um sinal dessa intensidade, convém segui-lo integralmente. Já o perfil de magnitude tem dispersão de $11,3\times$, distribuída entre várias camadas de sensibilidade intermediária; nele, a razão entre camadas é mais suscetível a ruído de medição, e amortecê-la funciona como regularização. O expoente β comporta-se, assim, como um parâmetro de confiança no perfil: quanto mais nítido o sinal, maior o β que compensa adotar.

5.5.4 Efeito sobre o Custo

Do ponto de vista da eficiência, a alocação guiada preserva integralmente o comportamento da estratégia de base, o que era esperado, uma vez que ela altera a distribuição do orçamento, não sua natureza. Na poda por magnitude, tamanho, FLOPs e tempo permanecem constantes, pelo mesmo motivo discutido na Seção 5.2. Na poda estruturada, o custo cai proporcionalmente: normalizando pelo *baseline* da própria sessão, a configuração com $\beta = 1$ registra tempo relativo de 0,659 e energia relativa de 0,674 em 50 %, e 0,419 e 0,384 em 90 % — valores equivalentes, dentro da variação entre sessões, aos 0,688 e 0,714 (50 %) e 0,445 e 0,413 (90 %) do escopo global.

O achado prático é, portanto, que **o ganho de qualidade da alocação guiada**

é gratuito em termos de custo: na faixa de 10 % a 50 % da poda estruturada, obtém-se uma perplexity de três a quatro vezes menor com essencialmente o mesmo tempo de inferência, o mesmo consumo de energia e o mesmo tamanho de modelo. É exatamente o deslocamento da fronteira entre compressão e qualidade que a segunda pergunta empírica do trabalho se propunha a investigar.

5.6 Síntese dos Resultados

Os experimentos permitem responder às duas perguntas que orientaram o trabalho.

À primeira — se a redução teórica de parâmetros se converte em redução real de consumo — a resposta é condicional à granularidade da poda. A poda não estruturada por magnitude reduz em 61 % os parâmetros não nulos sem alterar em nada o tempo de inferência, a energia, o tamanho em memória ou os FLOPs: na Tesla T4 utilizada, sem suporte de *hardware* para esparsidade, o ganho é puramente contábil. A poda estruturada, ao remover fisicamente cabeças de atenção e neurônios MLP, converte a mesma redução de 61 % em parâmetros em quedas de 56 % no tempo, 59 % na energia e 62 % nos FLOPs. Para o objetivo declarado deste trabalho — redução do consumo de energia —, apenas a poda estruturada é uma estratégia viável.

À segunda — se a alocação otimizada empurra a fronteira entre compressão e qualidade — a resposta é afirmativa, com escopo delimitado. As camadas do GPT-2 diferem entre si por mais de uma ordem de grandeza no custo de serem podadas, com a camada 0 respondendo isoladamente por cerca de três quartos da degradação da poda uniforme; essa informação, que nenhuma alocação ingênua utiliza, converte-se em ganho mensurável quando incorporada à distribuição do orçamento. Na poda estruturada — a estratégia que efetivamente reduz consumo — a alocação guiada reduz a perplexity por fatores de três a quatro na faixa de 10 % a 50 % de estruturas removidas, sem custo adicional de tempo, energia ou memória. O escopo da conclusão é delimitado por três fronteiras: o regime *one-shot*, a saturação da alocação em orçamentos acima de 60 %, e o fato de que o perfil de sensibilidade precisa ser levantado separadamente para cada estratégia de poda, por não ser transferível entre elas.

6 Considerações Finais

Referências

- BAHDANAU, D.; CHO, K.; BENGIO, Y. Neural machine translation by jointly learning to align and translate. In: *International Conference on Learning Representations (ICLR)*. [S.l.: s.n.], 2015. Citado na página 9.
- BENGIO, Y.; SIMARD, P.; FRASCONI, P. Learning long-term dependencies with gradient descent is difficult. *IEEE Transactions on Neural Networks*, v. 5, n. 2, p. 157–166, 1994. Citado na página 9.
- BLALOCK, D. et al. What is the state of neural network pruning? In: *Proceedings of Machine Learning and Systems (MLSys)*. [S.l.: s.n.], 2020. Citado 8 vezes nas páginas 15, 22, 25, 27, 29, 33, 34 e 38.
- COURTY, B. et al. *CodeCarbon: estimate and track carbon emissions from your computing*. 2023. Disponível em: <<https://github.com/mlco2/codecarbon>>. Acesso em: 17 abr. 2026. Citado 3 vezes nas páginas 21, 22 e 39.
- DENIL, M. et al. Predicting parameters in deep learning. In: *Advances in Neural Information Processing Systems (NeurIPS)*. [S.l.: s.n.], 2013. Citado na página 14.
- FANG, G. et al. Depgraph: Towards any structural pruning. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. [S.l.: s.n.], 2023. Citado 4 vezes nas páginas 25, 34, 38 e 39.
- FRANKLE, J.; CARBIN, M. The lottery ticket hypothesis: Finding sparse, trainable neural networks. In: *International Conference on Learning Representations (ICLR)*. [S.l.: s.n.], 2019. Citado 5 vezes nas páginas 17, 25, 27, 29 e 30.
- GOODFELLOW, I.; BENGIO, Y.; COURVILLE, A. *Deep Learning*. Cambridge, MA: MIT Press, 2016. Disponível em: <<https://www.deeplearningbook.org/>>. Citado 9 vezes nas páginas 2, 4, 5, 7, 20, 29, 30, 32 e 35.
- HAN, S. et al. Learning both weights and connections for efficient neural networks. In: *Advances in Neural Information Processing Systems (NeurIPS)*. [S.l.: s.n.], 2015. Citado 17 vezes nas páginas 15, 19, 22, 23, 24, 25, 26, 27, 33, 34, 35, 36, 37, 38, 39, 42 e 45.
- HE, K. et al. Deep residual learning for image recognition. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. [S.l.: s.n.], 2016. p. 770–778. Citado 8 vezes nas páginas 25, 27, 28, 29, 30, 32, 33 e 36.
- HENDRYCKS, D.; GIMPEL, K. Gaussian error linear units (GELUs). *arXiv preprint arXiv:1606.08415*, 2016. Citado na página 11.
- HINTON, G.; VINYALS, O.; DEAN, J. Distilling the knowledge in a neural network. *arXiv preprint arXiv:1503.02531*, 2015. Citado na página 14.
- HOCHREITER, S.; SCHMIDHUBER, J. Long short-term memory. *Neural Computation*, v. 9, n. 8, p. 1735–1780, 1997. Citado na página 9.

- JACOB, B. et al. Quantization and training of neural networks for efficient integer-arithmetic-only inference. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. [S.l.: s.n.], 2018. Citado na página 14.
- KRIZHEVSKY, A. *Learning Multiple Layers of Features from Tiny Images*. [S.l.], 2009. Citado na página 27.
- KRIZHEVSKY, A.; SUTSKEVER, I.; HINTON, G. E. Imagenet classification with deep convolutional neural networks. In: *Advances in Neural Information Processing Systems (NeurIPS)*. [S.l.: s.n.], 2012. Citado 3 vezes nas páginas 4, 27 e 32.
- LI, H. et al. Pruning filters for efficient convnets. In: *International Conference on Learning Representations (ICLR)*. [S.l.: s.n.], 2017. Citado 17 vezes nas páginas 19, 22, 23, 24, 25, 27, 29, 30, 33, 34, 35, 36, 37, 38, 42, 46 e 47.
- LUCCIONI, A. S.; VIGUIER, S.; LIGOZAT, A.-L. Estimating the carbon footprint of BLOOM, a 176B parameter language model. *arXiv preprint arXiv:2211.02001*, 2022. Citado na página 13.
- MICHEL, P.; LEVY, O.; NEUBIG, G. Are sixteen heads really better than one? In: *Advances in Neural Information Processing Systems*. [S.l.]: Curran Associates, Inc., 2019. v. 32. Citado 2 vezes nas páginas 11 e 47.
- NAIR, V.; HINTON, G. E. Rectified linear units improve restricted Boltzmann machines. In: *Proceedings of the 27th International Conference on Machine Learning (ICML)*. [S.l.: s.n.], 2010. Citado 2 vezes nas páginas 4 e 11.
- NVIDIA Corporation. *NVIDIA A100 Tensor Core GPU Architecture*. 2020. White paper. Descreve o suporte a esparsidade estruturada 2:4 na arquitetura Ampere. Citado na página 15.
- PASZKE, A. et al. Pytorch: An imperative style, high-performance deep learning library. In: *Advances in Neural Information Processing Systems (NeurIPS)*. [S.l.: s.n.], 2019. Citado 4 vezes nas páginas 18, 22, 31 e 39.
- PATTERSON, D. et al. Carbon emissions and large neural network training. *arXiv preprint arXiv:2104.10350*, 2021. Citado na página 13.
- RADFORD, A. et al. Language models are unsupervised multitask learners. *OpenAI Technical Report*, 2019. Citado 2 vezes nas páginas 11 e 12.
- RUMELHART, D. E.; HINTON, G. E.; WILLIAMS, R. J. Learning representations by back-propagating errors. *Nature*, v. 323, n. 6088, p. 533–536, 1986. Citado na página 6.
- SEVILLA, J. et al. Compute trends across three eras of machine learning. *arXiv preprint arXiv:2202.05924*, 2022. Citado na página 13.
- STRUBELL, E.; GANESH, A.; MCCALLUM, A. Energy and policy considerations for deep learning in nlp. In: *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics (ACL)*. [S.l.: s.n.], 2019. Citado na página 13.
- VASWANI, A. et al. Attention is all you need. In: *Advances in Neural Information Processing Systems (NeurIPS)*. [S.l.: s.n.], 2017. p. 5998–6008. Citado 3 vezes nas páginas 8, 9 e 10.

YANG, T.-J.; CHEN, Y.-H.; SZE, V. Designing energy-efficient convolutional neural networks using energy-aware pruning. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. [S.l.: s.n.], 2017. Citado 2 vezes nas páginas 26 e 39.

7 Apêndice

[TO DO]